



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт Информационных технологий

Кафедра Математического обеспечения и стандартизации
информационных технологий

Отчет по практической работе №4

по дисциплине «Проектирование и разработка мобильных приложений»

Выполнил:

Студент группы ИКБО-20-23

Кузнецов Л. А.

Проверил:

старший преподаватель кафедры
МОСИТ

Шешуков Л. С.

МОСКВА 2025 г.

СОДЕРЖАНИЕ

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ.....	3
1.1 Передача данных в вызвавший открытие activity с помощью Activity Result API	3
1.2 Фрагменты	10
1.3 Жизненный цикл фрагмента	12
1.4 Создание и размещение фрагментов.....	16
1.5 Навигация.....	22
2 ПРАКТИЧЕСКАЯ ЧАСТЬ	25
2.1 MainActivity	25
2.2 SecondActivity	26
2.3 ThirdActivity	27
2.4 Итоговый вид приложения.....	29
2.5 Создание проекта	34
2.5.1 StaticFragment	34
2.5.2 DynamicFragment.....	35
2.5.3 LastFragment.....	36
ЗАКЛЮЧЕНИЕ	40

1 ТЕОРИТИЧЕСКАЯ ЧАСТЬ

1.1 Передача данных в вызвавший открытие activity с помощью Activity Result API

В прошлых темах было рассмотрено как вызывать новую Activity, передавать ей некоторые данные и возвращаться обратно. Но мы можем не только передавать данные запускаемой Activity, но и ожидать от нее некоторого результата работы.

Ранее мы вызывали новую Activity с помощью метода `startActivity()`. Для получения же результата работы запускаемой Activity необходимо использовать Activity Result API.

Activity Result API предоставляет компоненты для регистрации, запуска и обработки результата другой Activity. Одним из преимуществ применения Activity Result API является то, что он отвязывает результат Activity от самой Activity. Это позволяет получить и обработать результат, даже если Activity, которая возвращает результат, в силу ограничений памяти или в силу других причин завершила свою работу.

Для регистрации функции, которая будет обрабатывать результат, Activity Result API предоставляет метод `registerForActivityResult()` (рисунок 1.1.1). Этот метод в качестве параметров принимает объекты `ActivityResultContract` и `ActivityResultCallback` и возвращает объект `ActivityResultLauncher`, который применяется для запуска другой activity.

```
ActivityResultLauncher<I> registerForActivityResult (  
    ActivityResultContractM<I, O> contract,  
    ActivityResultCallback<O> callback) ~
```

Рисунок 1.1.1 – Код метода `registerForActivityResult()`

`ActivityResultContract` определяет контракт: данные какого типа будут подаваться на вход и какой тип будет представлять результат. `ActivityResultCallback` представляет интерфейс с единственным методом

onActivityResult(), который определяет обработку полученного результата. Когда вторая activity закончит работу и возвратит результат, то будет как раз вызываться этот метод. Результат передается в метод в качестве параметра. При этом тип параметра должен соответствовать типу результата, определенного в ActivityResultContract (рисунок 1.1.2).

```
ActivityResultLauncher<Intent> mStartForResult =  
    registerForActivityResult(new  
        ActivityResultContracts.StartActivityForResult(),  
        new ActivityResultCallback<ActivityResult>() {  
            @Override  
            public void onActivityResult(ActivityResult result) {  
                // обработка result  
            }  
        });
```

Рисунок 1.1.2 – Шаблон для написания экземпляра класса ActivityResultLauncher

Класс ActivityResultContracts предоставляет ряд встроенных типов контрактов. Например, в листинге кода выше применяется встроенный тип ActivityResultContracts.StartActivityForResult, который в качестве входного объекта устанавливает объект Intent, а в качестве типа результата – тип ActivityResult.

Метод registerForActivityResult() регистрирует функцию-колбек и возвращает объект ActivityResultLauncher. С помощью этого мы можем запустить activity. Для этого у объекта ActivityResultLauncher вызывается метод launch() (рисунок 1.1.3).

```
ActivityResultLauncher<Intent> mStartForResult =  
    registerForActivityResult(new  
        ActivityResultContracts.StartActivityForResult(),  
        new ActivityResultCallback<ActivityResult>() {  
            @Override  
            public void onActivityResult(ActivityResult result) {  
                // обработка result  
            }  
        });  
Intent intent = new Intent();  
mStartForResult.launch(intent);
```

Рисунок 1.1.3 – Пример вызова метода launch

В метод `launch()` передается объект того типа, который определен объектом `ActivityResultContracts` в качестве входного.

Приведём пример подобного использования `ActivityResultContracts`.

Допустим у нас есть поле для ввода имени. После ввода имени на второй `activity`, мы можем выбрать запретить доступ, разрешить доступ или отменить операцию вовсе.

Определим в классе `MainActivity` запуск второй `activity` (рисунок 1.1.4).

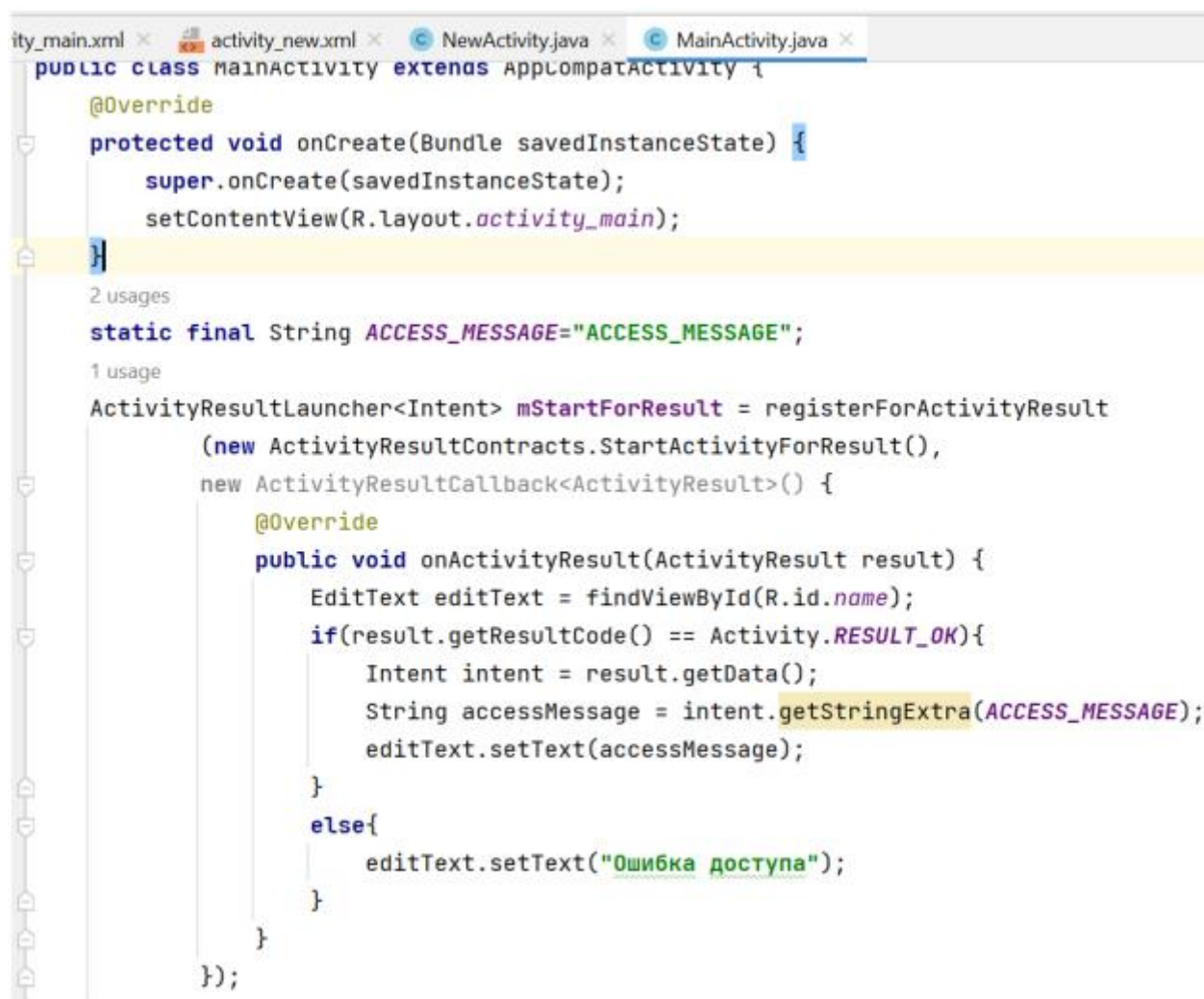


Рисунок 1.1.4 – Активация второй `activity` в основной

Прежде всего, мы определяем объект `ActivityResultLauncher`, с помощью которого будем запускать вторую `activity` и передавать ей данные. Объект `ActivityResultLauncher` типизируется типом `Intent`, так как объект этого типа будет передаваться в метод `launch()` при запуске второй `activity`.

Тип контракта определяется типом

ActivityResultContracts.StartActivityForResult, который и определяет тип Intent в качестве входного типа и тип ActivityResult в качестве типа результата.

Второй аргумент метода registerForActivityResult() – объект ActivityResultCallback типизируется типом результата – типом ActivityResult и определяет функцию-колбек onActivityResult(), которая получает результат и обрабатывает его. В данном случае обработка состоит в том, что мы выводим в текстовое поле ответ от второй activity.

При обработке мы проверяем полученный код результата (рисунок 1.1.5).



Рисунок 1.1.5 – Проверка полученного результата

В качестве результата, как правило, применяются встроенные константы Activity.RESULT_OK и Activity.RESULT_CANCELED.

На уровне условностей Activity.RESULT_OK означает, что activity успешно обработала запрос, а Activity.RESULT_CANCELED – что activity отклонила обработку запроса.

С помощью метода getData() результата получаем переданные из второй activity данные в виде объекта Intent (рисунок 1.1.6).

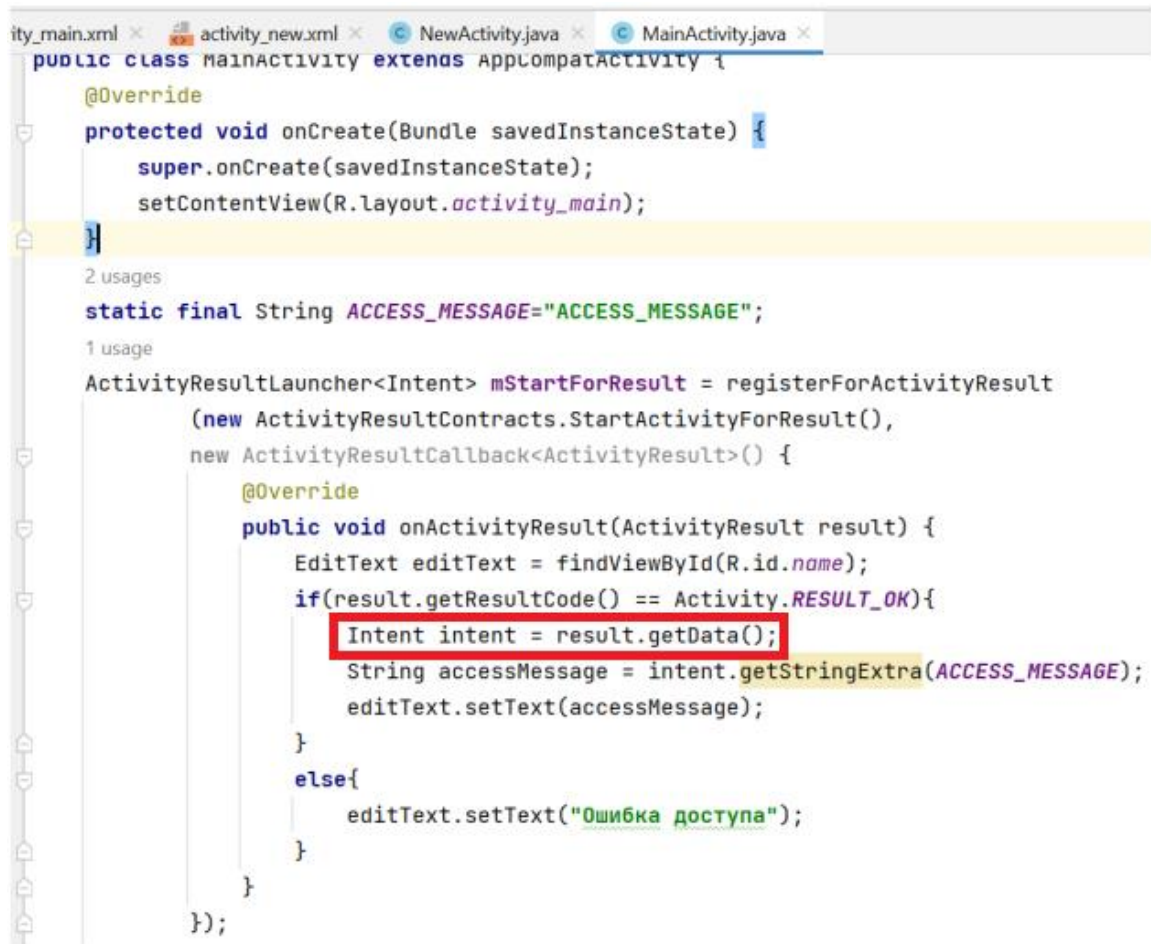


Рисунок 1.1.6 – Получение данных из второй activity

Далее извлекаем из Intent строку, которая имеют ключ ACCESS_MESSAGE, и выводим ее в текстовое поле.

Таким образом, мы определили объект ActivityResultLauncher.

Далее в обработчике нажатия onClick с помощью этого объекта запускаем вторую activity (рисунок 1.1.7).


```

public void onNextActivity(View view){
    //Получаем введенное имя, которое будем проверять
    EditText textName = findViewById(R.id.name);
    String name = textName.getText().toString();
    Intent intent = new Intent( packageContext: this, NewActivity.class);
    intent.putExtra( name: "name", name);
    startActivityForResult.launch(intent);
}

```

Рисунок 1.1.7 – Метод запуска второй activity

В обработчике нажатия кнопки onClick() получаем введенное в текстовое поле имя, добавляем его в объект Intent и запускаем SecondActivity с помощью метода launch().

После этого определим логику работы второй Activity (рисунок 1.1.8).

```

@OVERRIDE
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_new);
    Bundle arguments = getIntent().getExtras();
    if (arguments!=null){
        EditText editText = findViewById(R.id.name2);
        String name = arguments.get("name").toString();
        editText.setText(name);
    }
}

1 usage
public void onAccessClickOk(View view) { sendMessage("Доступ разрешен"); }

1 usage
public void onAccessDeny(View view) { sendMessage("Доступ запрещен"); }

2 usages
private void sendMessage(String message){
    Intent intent = new Intent();
    intent.putExtra(MainActivity.ACCESS_MESSAGE, message);
    setResult(RESULT_OK, intent);
    finish();
}

1 usage
public void onCancelClick(View view) {
    setResult(RESULT_CANCELED);
    finish();
}

```

Рисунок 1.1.8 – Описание логики работы второй activity

Две кнопки вызывают метод `sendMessage()`, в который передают отправляемый ответ. Это и будет то сообщение, которое получить MainActivity в методе `onActivityResult`.

Для возврата результата необходимо вызвать метод `setResult()`, в который передается два параметра:

- числовой код результата;
- отправляемые данные.

После вызова метода `setResult()` нужно вызвать метод `finish`, который уничтожит текущую activity. `finish` – метод, при вызове которого система Android завершает работу текущей activity и удаляет её из Activity Stack (коллекция приостановленных системой activities, которые можно легко возобновить).

Одна кнопка вызывает обработчик `onCancelClick()`, в котором передается в `setResult` только код результата - `RESULT_CANCELED`.

То есть условно говоря, мы получаем в SecondActivity введенное в MainActivity имя и с помощью нажатия определенной кнопки возвращаем некоторый результат в виде сообщения.

В зависимости от нажатой кнопки на SecondActivity мы будем получать разные результаты в MainActivity (рисунок 1.1.9).

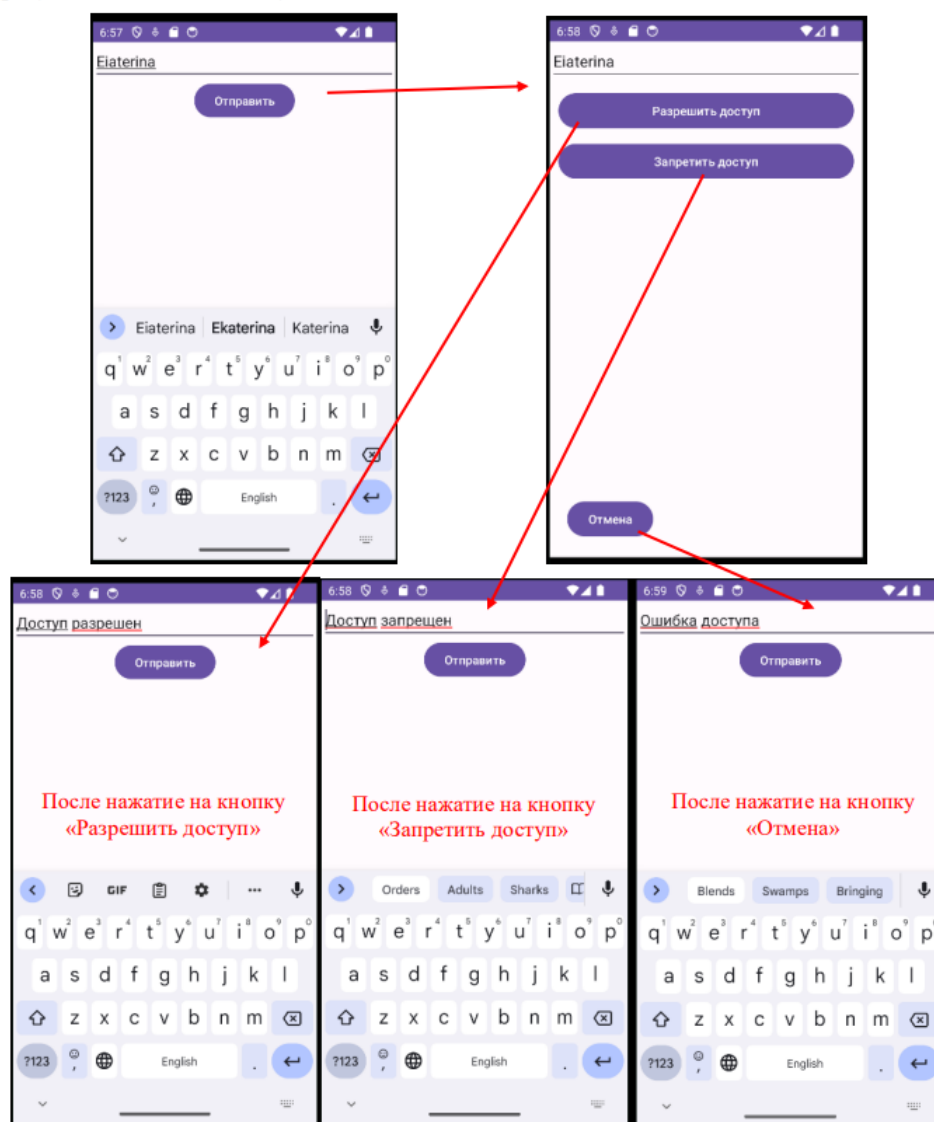


Рисунок 1.1.9 – Отображение различных вариантов развития событий в MainActivity

1.2 Фрагменты

Организация приложения на основе нескольких activity не всегда может быть оптимальной. Мир ОС Android довольно сильно фрагментирован и состоит из многих устройств. И если для мобильных аппаратов с небольшими экранами взаимодействие между разными activity выглядит довольно неплохо, то на больших экранах - планшетах, телевизорах окна activity смотрелись бы не очень в силу большого размера экрана. Собственно, поэтому и появилась концепция фрагментов.

Фрагмент представляет кусочек визуального интерфейса приложения, который может использоваться повторно и многократно (рисунок 1.2.1). У фрагмента может быть собственный файл layout, у фрагментов есть свой собственный жизненный цикл. Фрагмент существует в контексте activity и имеет свой жизненный цикл, вне activity обособлено он существовать не может. Каждая activity может иметь несколько фрагментов.

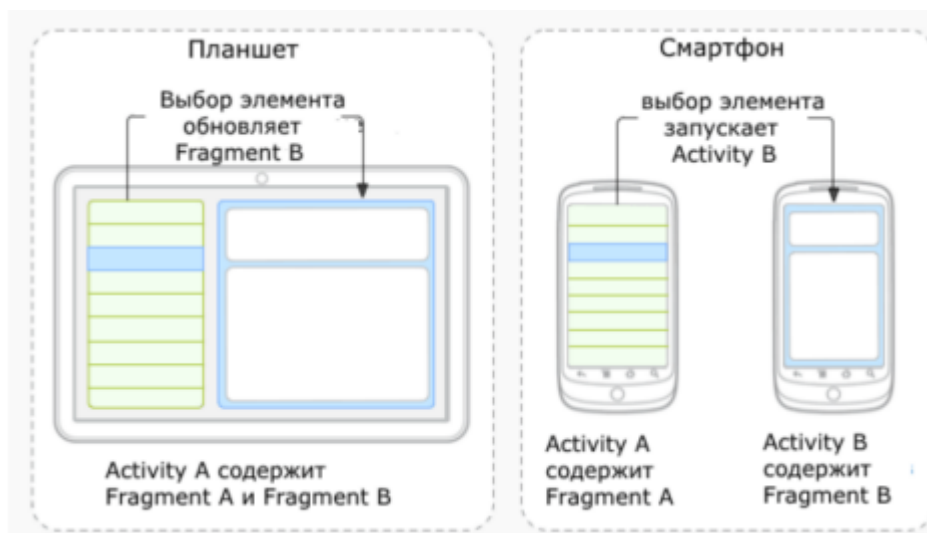


Рисунок 1.2.1 – Пример использования фрагментов

В Android Studio выделяют такие популярные виды фрагментов, как:

- UI-фрагменты (View Fragments): наиболее распространённый тип фрагментов, которые используются для отображения пользовательского интерфейса. Они имеют собственный жизненный цикл и могут управлять своим макетом (layout). Пример: фрагмент, отображающий список элементов, или фрагмент с деталями элемента;
- диалоговые фрагменты (Dialog Fragments): используются для отображения диалоговых окон. Позволяют создавать кастомные диалоги (небольшие окна, в которых пользователь может принять решение или ввести дополнительную информацию), которые могут быть повторно использованы в разных activities. Пример: диалог подтверждения действия или диалог с настройками;
- фрагменты-списки (List Fragments): предназначены для отображения списка элементов (например, с

использованием ListView или RecyclerView). Упрощают работу со списками, так как уже содержат встроенные методы для управления ими.

Фрагменты в Android представляют собой модульный сегмент пользовательского интерфейса в activity. Они используются для создания более гибких и адаптивных интерфейсов. Фрагменты могут быть добавлены, удалены, заменены или изменены в activity во время выполнения приложения. Это обеспечивает более динамичное и адаптивное взаимодействие с пользователем.

Среди основных характеристик фрагментов можно выделить:

- модульность: фрагменты позволяют разделить activity на несколько компонентов, каждый из которых имеет свой собственный жизненный цикл и обрабатывает свои собственные вводы. Это упрощает управление различными частями интерфейса пользователя и повторное использование компонентов в разных activities;
- адаптивность: фрагменты помогают создавать интерфейсы, которые легко адаптируются к различным размерам экрана и ориентациям, что особенно важно для устройств с разными размерами экранов, таких как телефоны и планшеты;
- управление жизненным циклом: каждый фрагмент имеет свой собственный жизненный цикл, но он тесно связан с жизненным циклом своей хост-activity. Это позволяет фрагментам управлять своим состоянием и поведением в зависимости от состояния activity.

1.3 Жизненный цикл фрагмента

Жизненный цикл фрагмента в Android представляет собой последовательность состояний, через которые проходит фрагмент во время своего существования (рисунок 1.3.1). Эти состояния позволяют управлять поведением фрагмента на различных этапах его взаимодействия с пользователем и системой.

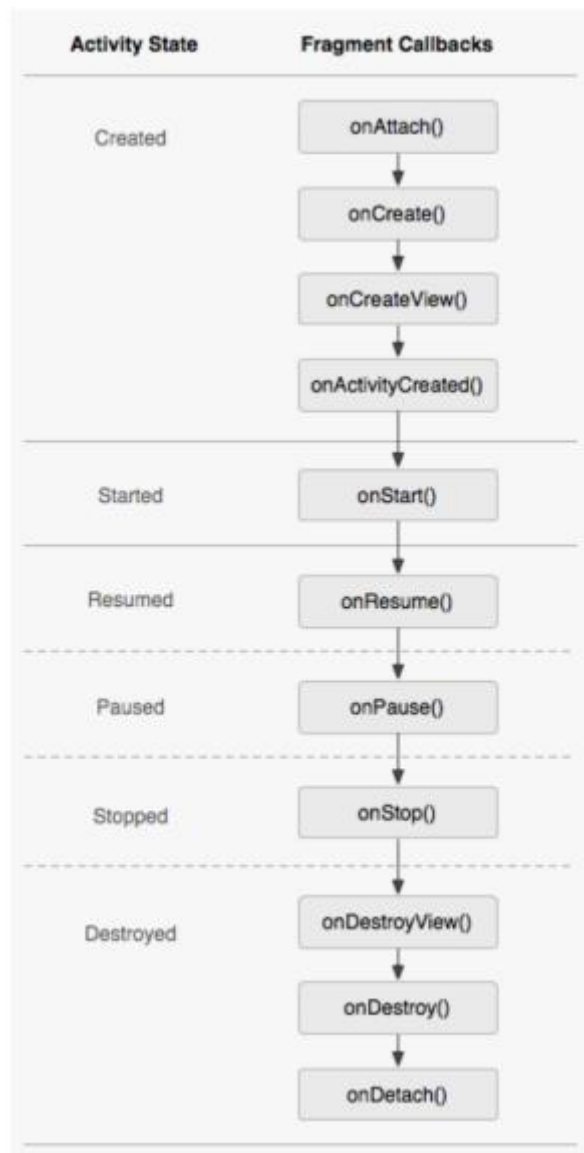


Рисунок 1.3.1 – Жизненный цикл фрагмента

Привязка к activity: когда фрагмент привязывается к activity, вызывается метод `onAttach()`. Это означает, что фрагмент теперь ассоциирован с activity, и разработчик может взаимодействовать с ней.

Создание фрагмента: Вызов метода `onCreate()` означает, что создается объект фрагмента (рисунок 1.3.2).. Здесь можно инициализировать компоненты, необходимые фрагменту для функционирования, но при этом не связанные с графическим интерфейсом.

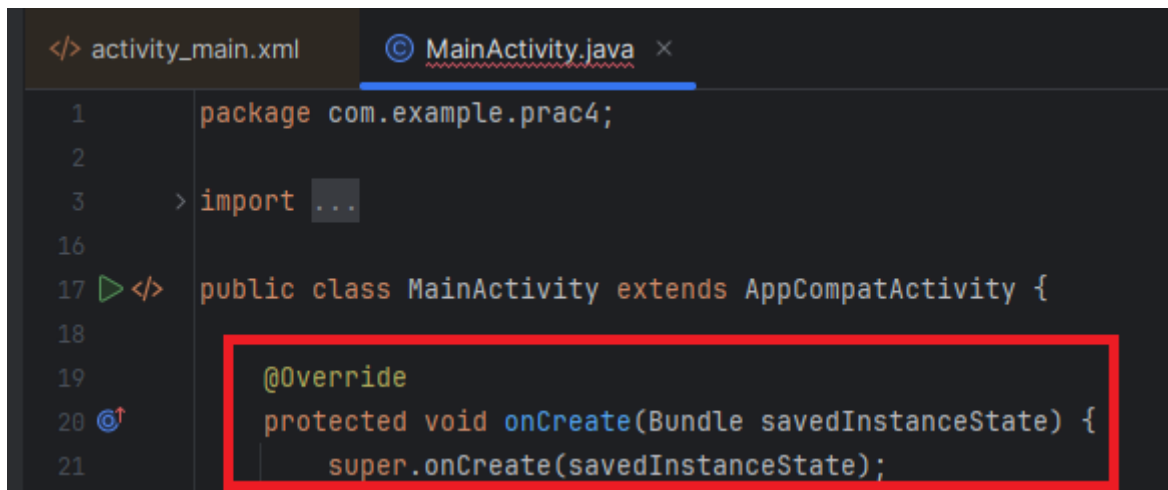


Рисунок 1.3.2 – Определение метода onCreate

Создание представления фрагмента: метод onCreateView() вызывается для создания представления фрагмента (рисунок 1.3.3). Здесь загружается макет, определяет пользовательский интерфейс фрагмента. После создания представления оно возвращается системе для отображения.

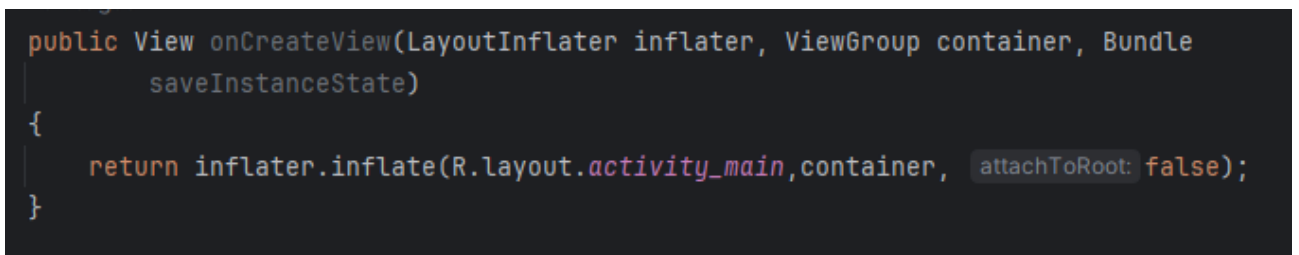


Рисунок 1.3.3 – Определение метода onCreateView

Первый параметр – объект LayoutInflater позволяет получить содержимое ресурса layout и передать его во фрагмент.

Второй параметр – объект ViewGroup представляет контейнер, в которой будет загружаться фрагмент.

Третий параметр – объект Bundle представляет состояние фрагмента. (Если фрагмент загружается первый раз, то равен null).

На выходе метод возвращает созданное с помощью LayoutInflater представление в виде объекта View – собственно представление фрагмента.

Также идёт обращение к R.layout, где layout - это автоматически генерируемый класс, который содержит ссылки на все XML-файлы макетов (layout files), расположенные в папке res/layout проекта.

Activity фрагмента: после создания представления, метод `onActivityCreated()` вызывается, когда activity, к которой привязан фрагмент, полностью создана. Это хорошее место для выполнения финальных инициализаций, которые зависят от activity, например, настройка компонентов интерфейса или получение данных.

Запуск фрагмента: когда фрагмент становится видимым для пользователя, вызывается метод `onStart()` (рисунок 1.3.4). Здесь можно запускать анимации или выполнять задачи, которые должны быть видны пользователю.

```
@Override
public void onStart() {
    super.onStart();
    // Код, который должен выполняться, когда фрагмент становится видимым
    Log.d("FragmentLifecycle", "onStart called");

    // Пример: обновление данных во фрагменте
    updateFragmentUI();
}
```

Рисунок 1.3.4 – Пример использования `onStart`

Возобновление фрагмента: в этом состоянии фрагмент полностью активен и взаимодействует с пользователем. Метод `onResume()` вызывается, когда фрагмент готов к пользовательскому взаимодействию.

Приостановка фрагмента: когда фрагмент перестает взаимодействовать с пользователем, система вызывает метод `onPause()`. Это происходит, например, при переключении на другой фрагмент или activity.

Остановка фрагмента: метод `onStop()` вызывается, когда фрагмент больше не виден пользователю. В этом состоянии можно освободить ресурсы, которые не нужны, пока фрагмент не активен.

Уничтожение представления фрагмента: Перед удалением фрагмента из activity или при его замене, метод `onDestroyView()` вызывается для очистки ресурсов, связанных с пользовательским интерфейсом фрагмента.

Уничтожение фрагмента: На этом этапе вызывается метод `onDestroy()`, который сигнализирует о том, что объект фрагмента скоро будет уничтожен. Это последний шанс для освобождения оставшихся ресурсов.

Открепление фрагмента от `activity`: Последний этап в жизненном цикле фрагмента — это его открепление от `activity`. Метод `onDetach()` вызывается, когда фрагмент отсоединяется от `activity`, что означает полное удаление фрагмента.

1.4 Создание и размещение фрагментов

Можно добавить по отдельности класс Java, который представляет фрагмент, и файл `xml` для хранения в нем разметки интерфейса, который будет использовать фрагмент, однако Android Studio представляет готовый шаблон для добавления фрагмента. Этот способ и будет использован в данной работе.

Принцип создания новых фрагментов схож с созданием `activity`. Нужно выбрать модуль "app", затем в верхней панели выбрав "File" → "New" → "Fragment" и выбрать тип `activity`, например "Fragment (Blank)" либо нажать правой кнопкой мыши на "java" → "New" → "Fragment" (рисунок 1.4.1), после чего выбрать название фрагмента (рисунок 1.4.2) и как в случае с `activity` будут созданы файлы класса и разметки.

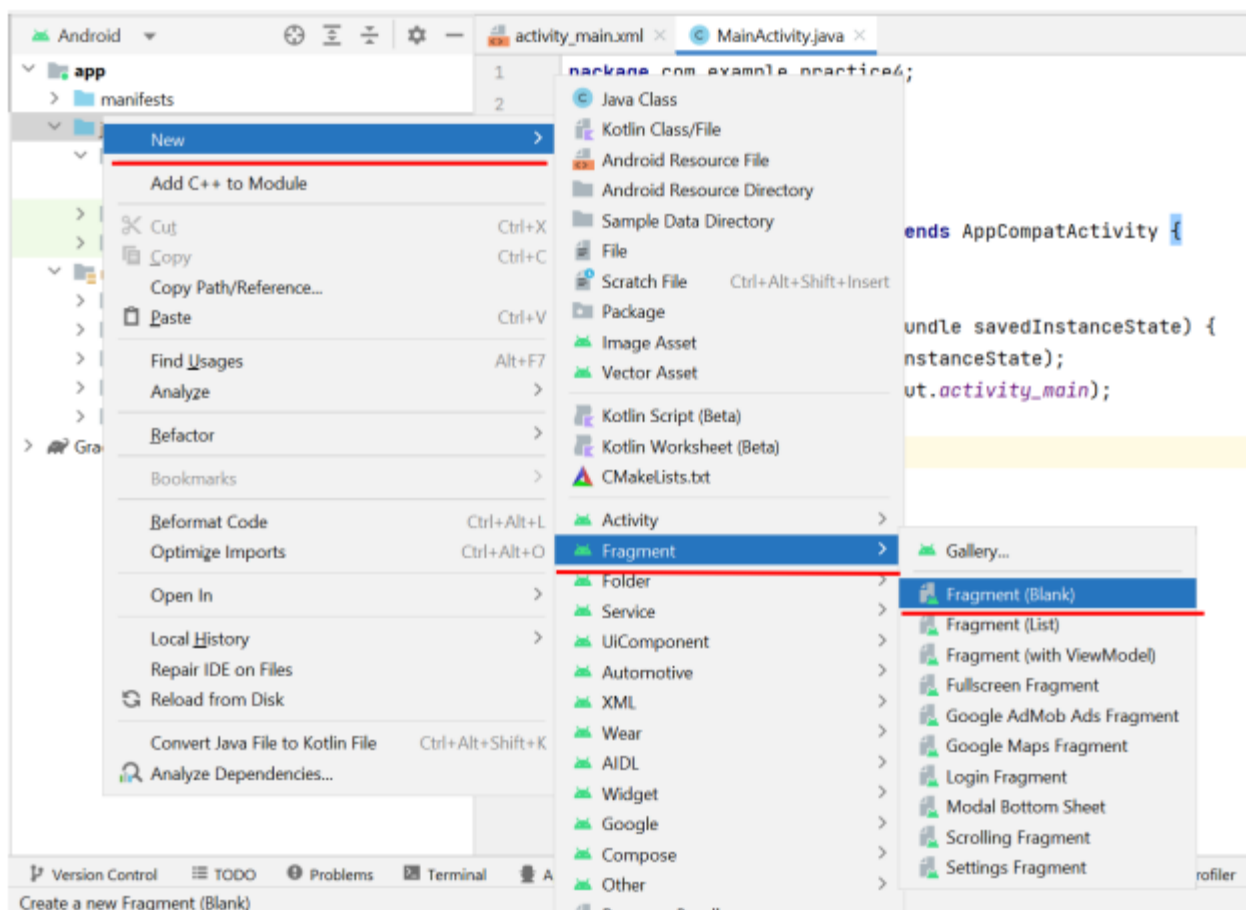


Рисунок 1.4.1 – Создание класса Java, представляющего фрагмент

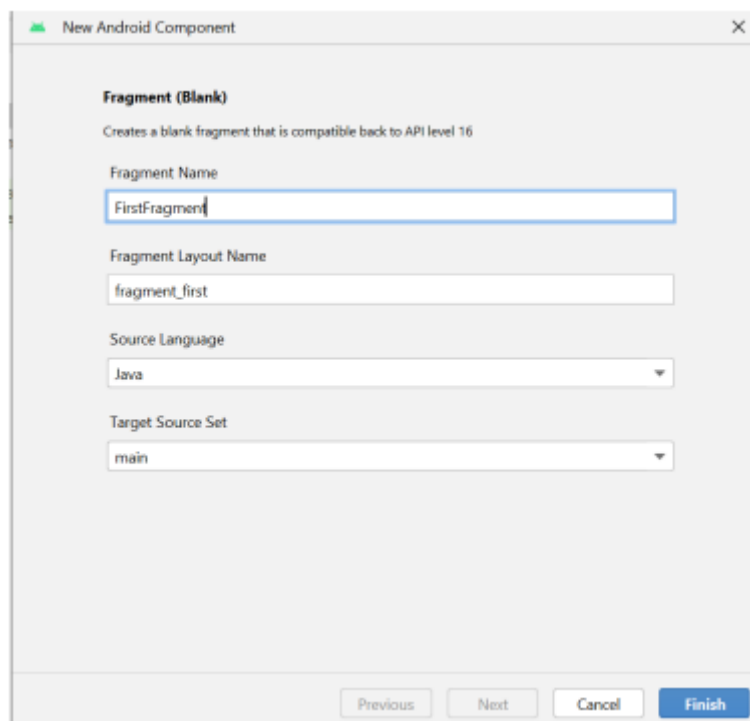


Рисунок 1.4.2 – Описание класса-фрагмента

После создания фрагмента, класс для реализации фрагмента и xml-файл для разметки будут созданы автоматически.

Класс фрагмента должен наследоваться от класса `Fragment` (рисунок 1.4.3). Чтобы указать, что фрагмент будет использовать определенный xml-файл layout, идентификатор ресурса layout передается в вызов конструктора родительского класса (то есть класса `Fragment`).

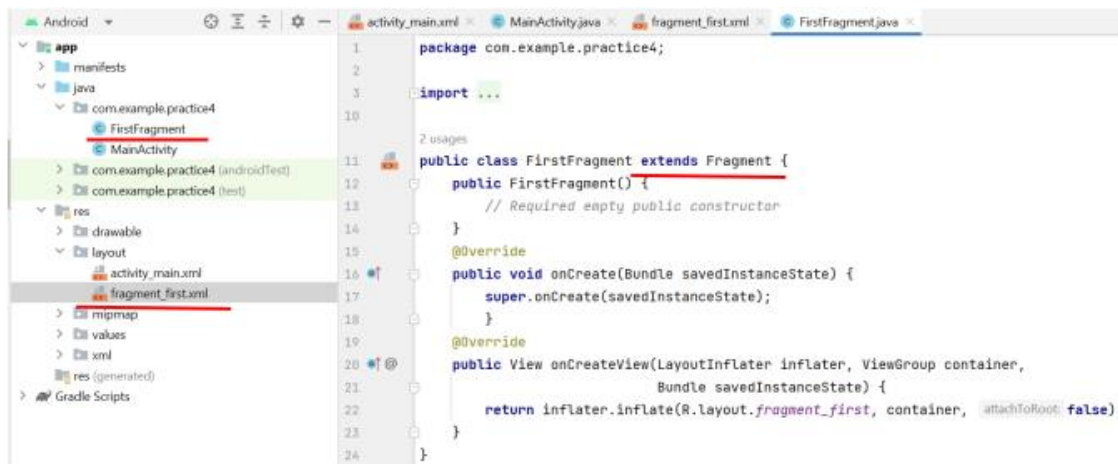


Рисунок 1.4.3 – Наследование класса-фрагмента

Теперь после создания фрагмента его необходимо разместить на `activity`. Сделать это можно тремя способами: статически, динамически и через элемент `fragmentContainerView`.

Для размещения фрагмента статическим способом необходимо добавить в XML файл `activity` элемент `fragment` (рисунок 1.4.4).

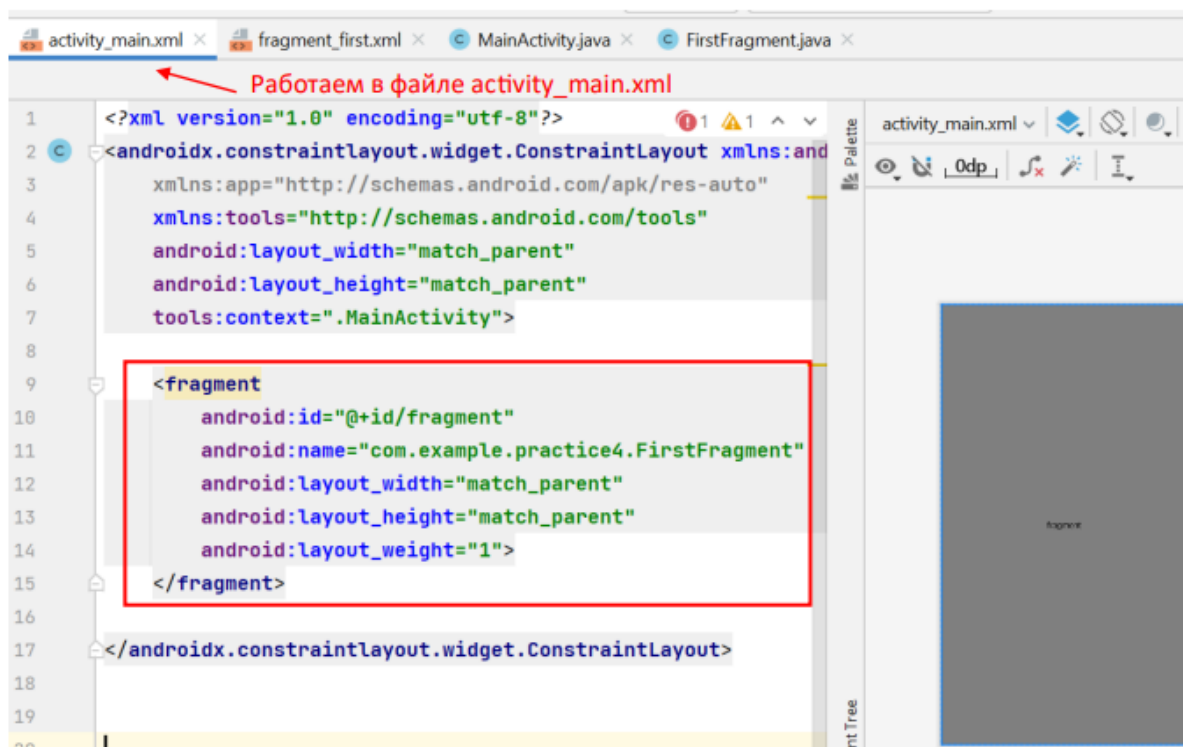


Рисунок 1.4.4 – Размещение фрагмента статическим способом

Атрибут `android:name` указывает на полное имя класса фрагмента с учетом пакета, который будет использоваться.

В качестве примера во фрагменте было определено текстовое поле с некоторым текстовым значением (рисунок 1.4.5).

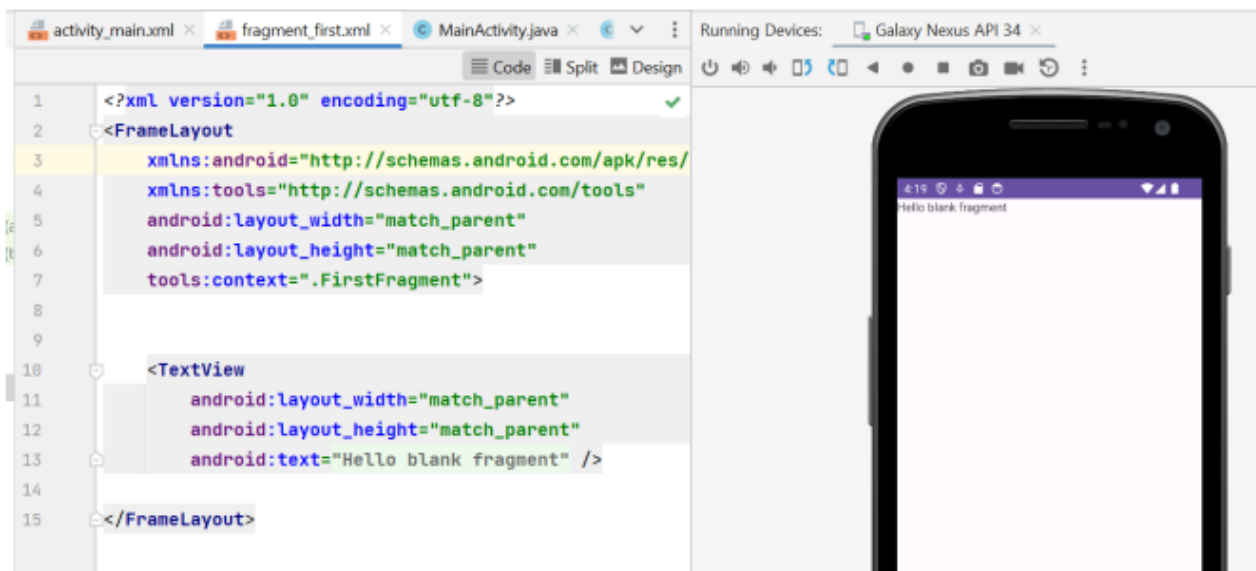


Рисунок 1.4.5 – Размещение фрагмента с текстовым полем

Теперь данный фрагмент будет показываться при запуске приложения.

Главным недостатком такого размещения фрагмента заключается в том, что статические фрагменты тесно связаны с жизненным циклом `activity`, что может усложнить управление их состоянием. Например, сохранение и восстановление Работаем в файле `activity_main.xml` состояния фрагмента при пересоздании `activity` может потребовать дополнительной логики.

Все преимущества фрагментов раскрываются при их динамическом изменении в процессе работы приложения. Фрагменты, при динамическом размещении, управляются аналогично обычным `View` элементам. Для облегчения управления, каждый фрагмент размещается в отдельном контейнере. Обычно для этой цели выбираются контейнеры типа `FrameLayout` (рисунок 1.4.6).

```
<FrameLayout
    android:id="@+id/fragment"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

Рисунок 1.4.6 - Описание контейнера типа **FrameLayout**

Далее в классе `activity` необходимо «разместить» данный фрагмент, используя метод класса `"FragmentManager"` — `"getSupportFragmentManager"` (рисунок 1.4.7).

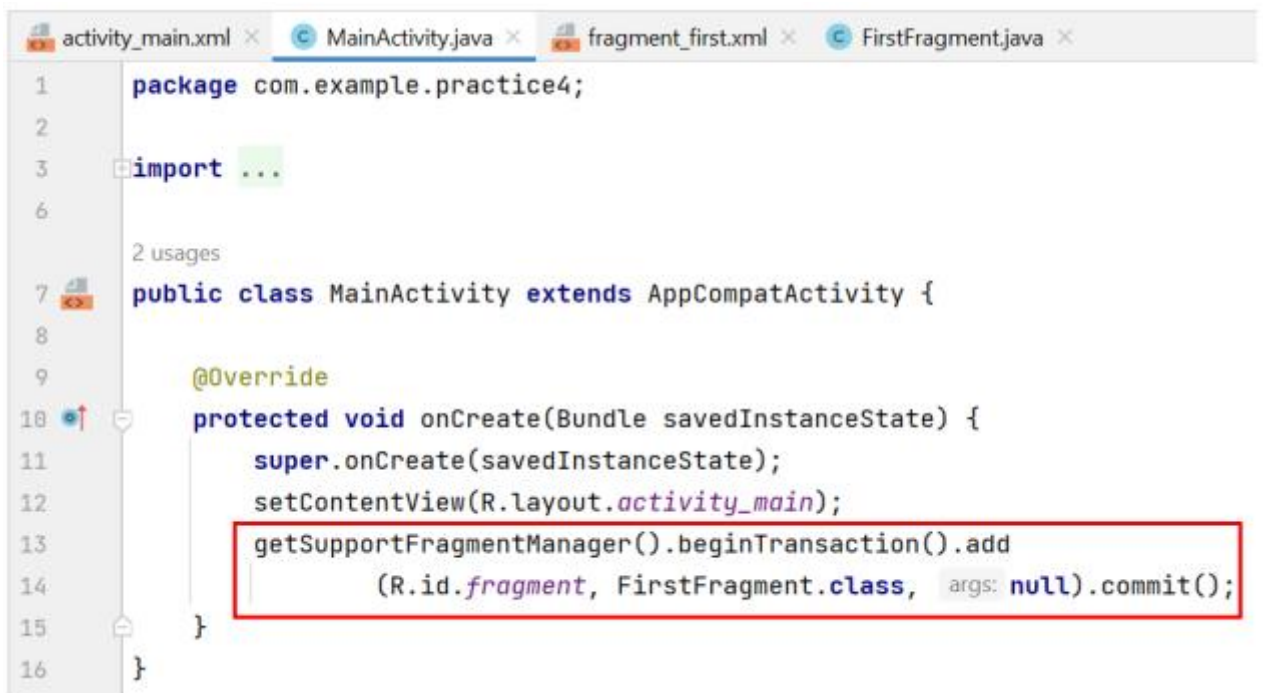


Рисунок 1.4.7 – Размещение фрагмента через код

Метод `getSupportFragmentManager()` возвращает объект `FragmentManager`, который управляет фрагментами.

Объект `FragmentManager` с помощью метода `beginTransaction()` создает объект `FragmentTransaction`.

`FragmentTransaction` выполняет два метода: `add()` и `commit()`. Метод `add()` добавляет фрагмент.

И метод `commit()` подтверждает и завершает операцию добавления.

Последний способ размещения фрагментов является ключевым элементом современного подхода к управлению фрагментами в приложении. `FragmentManager` — это специализированный контейнер для фрагментов, который предлагает улучшенную замену `FrameLayout` при

работе с фрагментами. Он более оптимизирован для работы с FragmentManager и предоставляет дополнительные возможности и преимущества.

Для добавления фрагмента применяется элемент `FragmentManager` и предоставляет дополнительные возможности и преимущества. По сути, `FragmentManager` представляет объект `View`, который расширяет класс `FrameLayout` и предназначен специально для работы с фрагментами. Собственно, кроме фрагментов он больше ничего содержать не может.

Для его использования нужно разместить данный элемент в XML файле `activity` и в открывшемся окне выбрать нужный фрагмент, и он автоматически будет размещен внутри `FragmentManager` (рисунки 1.4.8-1.4.9).

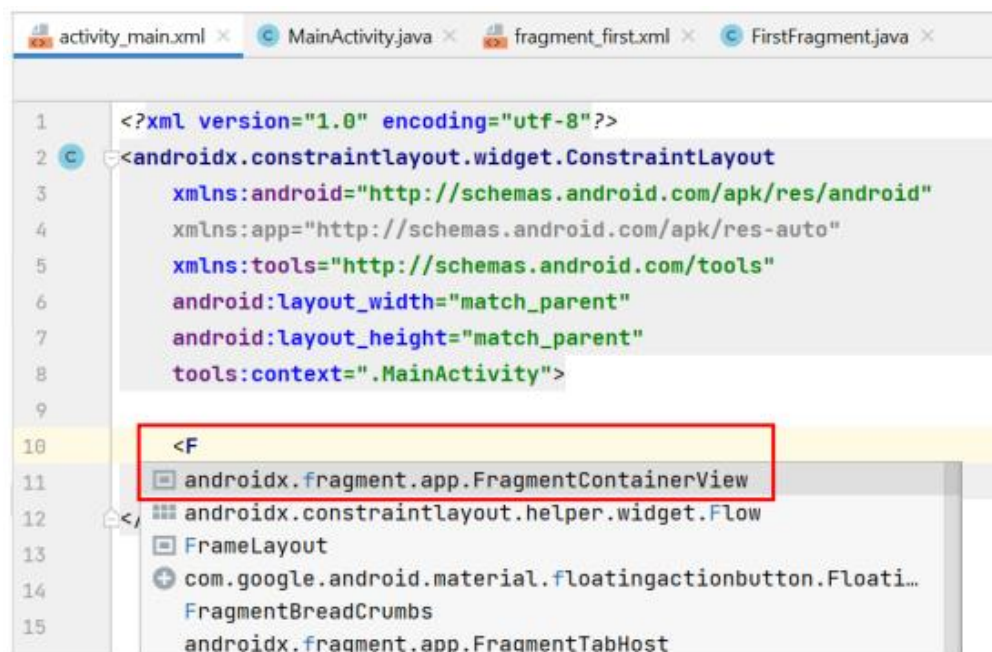


Рисунок 1.4.8 – Часть 1 размещения фрагмента в XML файле `activity`

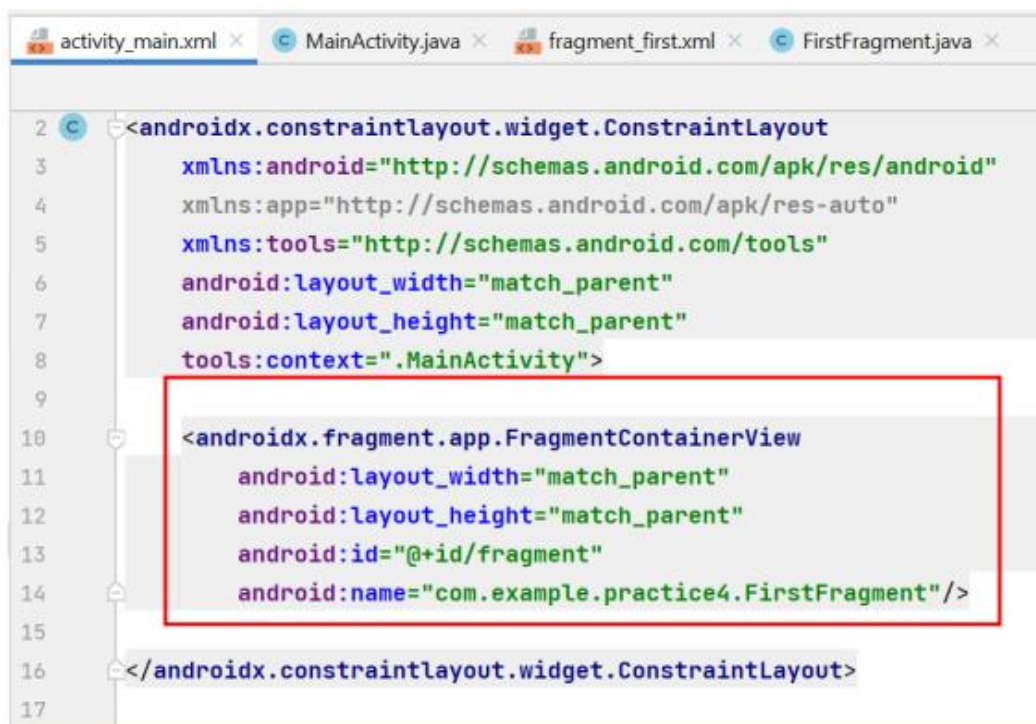


Рисунок 1.4.9 – Часть 2 размещения фрагмента в XML файле activity

1.5 Навигация

Навигация между фрагментами осуществляется с помощью `FragmentManager`, который управляет операциями, такими как добавление, удаление или замена фрагментов в контейнере. Ключевым понятием в навигации между фрагментами является транзакция. Транзакция фрагмента — это серия действий, которые выполняются вместе. Эти действия могут включать добавление, удаление и замену фрагментов, а также добавление их в стек возврата, чтобы пользователь мог вернуться обратно, используя кнопку «Назад».

Для управления навигацией между фрагментами можно использовать программные методы, такие как вызовы `"replace"` и `"commit"` класса `FragmentManager`. Для этого нужно создать объект того фрагмента куда планируется осуществить перемещение и добавить его в метод `replace` и сохранить изменения через метод `commit` (рисунок 1.5.1).


```

FragmentManager fragmentManager = getSupportFragmentManager();
1 usage
FirstFragment fragment1 = new FirstFragment();
2 usages
FragmentManager.beginTransaction()
fragmentTransaction.replace(R.id.container, fragment1, "fragment1");
fragmentTransaction.commit();

```

Рисунок 1.5.1 – Реализация навигации между фрагментами

Где FirstFragment – это название java класса первого фрагмента, а container – идентификатор FrameLayout, который расположен в разметки activity_main.

Тогда при нажатии на кнопку будут меняться используемые фрагменты (рисунки 1.5.2-1.5.3).



Рисунок 1.5.2 – Часть 1 отображения используемых фрагментов



Рисунок 1.5.3 - Часть 2 отображения используемых фрагментов

2 ПРАКТИЧЕСКАЯ ЧАСТЬ

2.1 MainActivity

После создания проекта обратимся к файлу разметки `activity_main.xml` и опишем все необходимые элементы согласно заданию (рисунок 2.1.1).

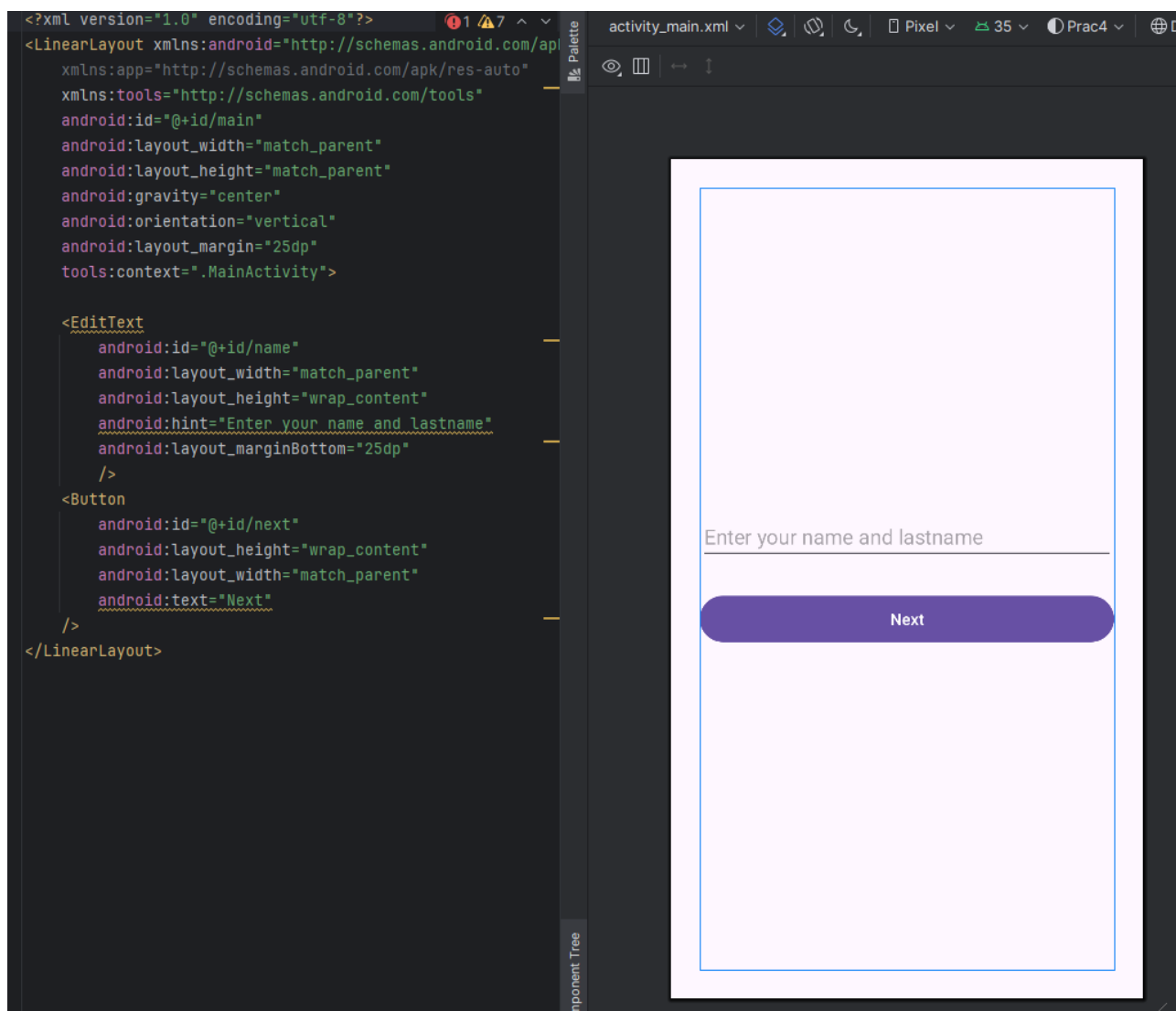


Рисунок 2.1.1 – Графический и кодовый вид файла разметки `activity_main.xml`

На рисунке 2.1.1 отображены два элемента интерфейса: `EditText`, предназначенный для ввода имени и фамилии пользователя, а также `Button`, необходимый для перехода на следующую `activity`.

При этом была реализована логика перехода на другую `activity` с передачей в неё строковых данных (рисунок 2.1.2).

```

<> public class SecondActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( this::enableEdgeToEdge );
        setContentView(R.layout.activity_second);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
        String date = (String) getIntent().getSerializableExtra( name: "date");
        if (date != null){
            Toast.makeText(getApplicationContext(), text: "Время занятия успешно передано", Toast.LENGTH_SHORT).show();
        }
        TextView name = findViewById(R.id.nameView);
        TextView lastName = findViewById(R.id.lastNameView);
        String names = (String) getIntent().getSerializableExtra( name: "name&lastName");
        String[] namesArray = names.split( regex: " " );
        name.setText(namesArray[0]);
        lastName.setText(namesArray[1]);

        Button nextButton = findViewById(R.id.next);
        nextButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                onClick(v);
            }
        });

        1 usage
        public void onClick(View view){
            Intent intent = new Intent( packageContext: this, ThirdActivity.class);
            startActivity(intent);
        }
    }
}

```

Рисунок 2.1.2 – Реализация логики файла разметки activity_main.xml

2.2 SecondActivity

Далее создадим второй файл разметки под названием activity_second.xml, реализовав необходимый функционал (рисунок 2.2.1).

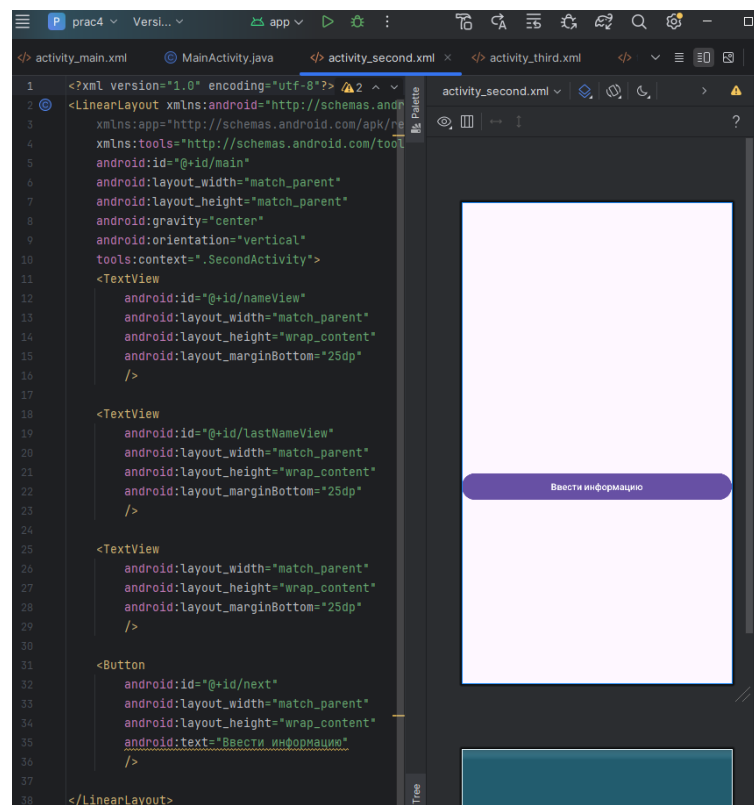


Рисунок 2.2.1 - Графический и кодовый вид файла разметки activity_second.xml

На рисунке 2.2.1 отображены 4 элемента разметки: 3 TextView, предназначенные для отображения входных данных, а также Button, необходимая для перехода на другую activity.

При этом была реализована логика файла разметки activity_second.xml (рисунок 2.2.2).

```
></> public class SecondActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable( $this$enableEdgeToEdge: this);
        setContentView(R.layout.activity_second);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });
        String date = (String) getIntent().getSerializableExtra( name: "date");
        if (date != null){
            Toast.makeText(getApplicationContext(), text: "Время занятия успешно передано", Toast.LENGTH_SHORT).show();
        }
        TextView name = findViewById(R.id.nameView);
        TextView lastName = findViewById(R.id.lastNameView);
        String names = (String) getIntent().getSerializableExtra( name: "name&LastName");
        String[] namesArray = names.split( regex: " ");
        name.setText(namesArray[0]);
        lastName.setText(namesArray[1]);

        Button nextButton = findViewById(R.id.next);
        nextButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                onClick(v);
            }
        });
    }

    1 usage
    public void onClick(View view){
        Intent intent = new Intent( packageContext: this, ThirdActivity.class);
        startActivity(intent);
    }
}
```

Рисунок 2.2.2 – Реализация логики файла разметки activity_second.xml

2.3 ThirdActivity

Далее был создан третий файл разметки – activity.third.xml, с сопутствующими ему по заданию элементами интерфейса (рисунок 2.3.1).

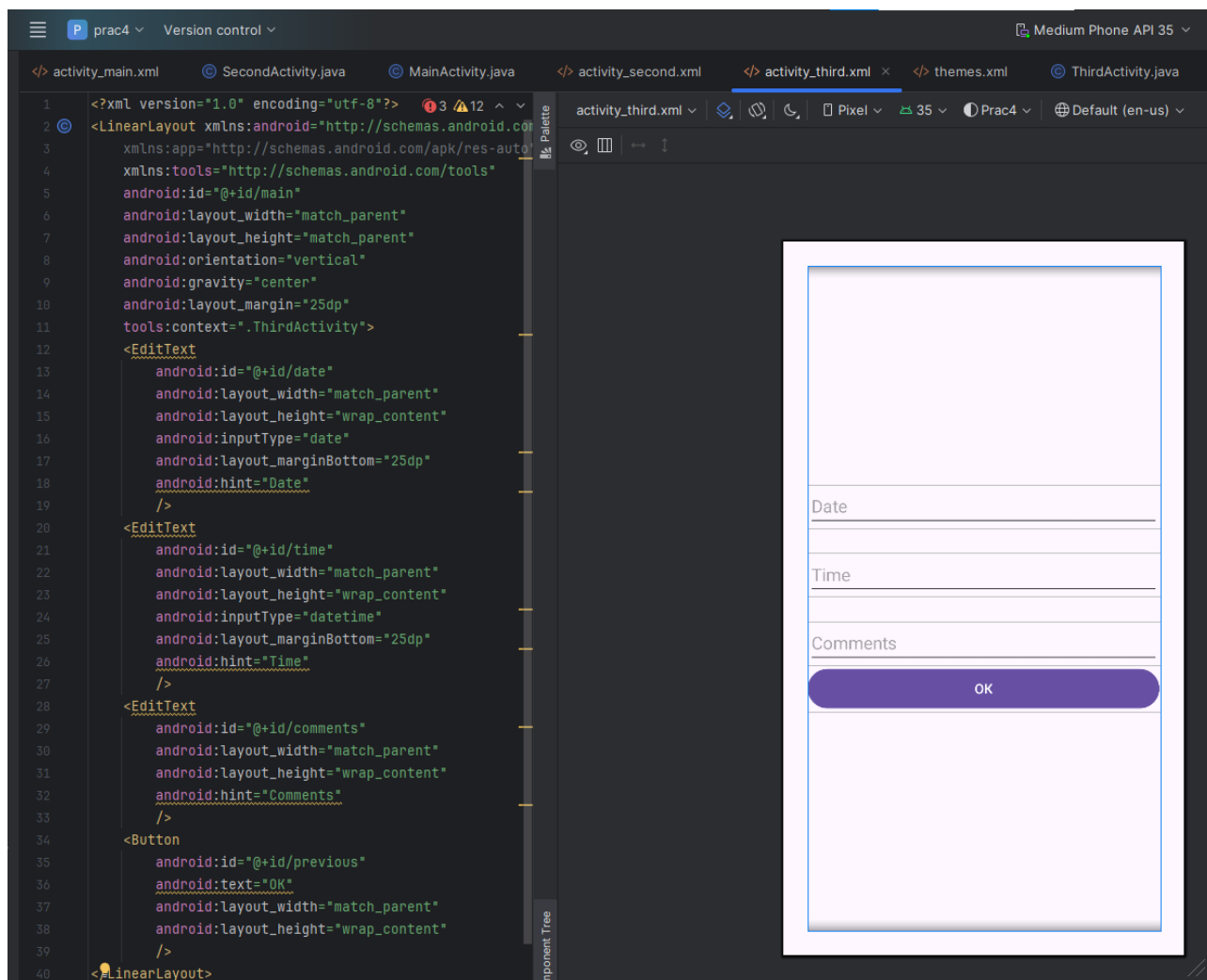


Рисунок 2.3.1 – Графический и кодовый вид файла разметки activity_third.xml

На рисунке 2.3.1 представлены такие элементы интерфейса, как 3 экземпляра `EditText`, предназначенные для ввода даты, времени и комментариев соответственно, а также `Button`, необходимый для перехода на предыдущую `activity`.

При этом была реализована логика файла разметки `activity_third.xml` (рисунок 2.3.2).

```

public class ThirdActivity extends AppCompatActivity {

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        EdgeToEdge.enable(this);
        setContentView(R.layout.activity_third);
        ViewCompat.setOnApplyWindowInsetsListener(findViewById(R.id.main), (v, insets) -> {
            Insets systemBars = insets.getInsets(WindowInsetsCompat.Type.systemBars());
            v.setPadding(systemBars.left, systemBars.top, systemBars.right, systemBars.bottom);
            return insets;
        });

        Button previousButton = findViewById(R.id.previous);
        previousButton.setOnClickListener(new View.OnClickListener() {
            @Override
            public void onClick(View v) {
                onClick(v);
            }
        });
    }

    1 usage
    public void onClick(View view){
        Intent intent = new Intent(packageContext: this, SecondActivity.class);
        EditText date = findViewById(R.id.date);
        EditText time = findViewById(R.id.time);
        EditText comments = findViewById(R.id.comments);
        intent.putExtra(name: "date", date.getText().toString());
        intent.putExtra(name: "time", time.getText().toString());
        intent.putExtra(name: "comments", comments.getText().toString());
        startActivity(intent);
    }
}

```

Рисунок 2.3.2 - Реализация логики файла разметки activity_third.xml

2.4 Итоговый вид приложения

Теперь отобразим конечный вид полученного приложения на рисунках 2.4.1 – 2.4.4.

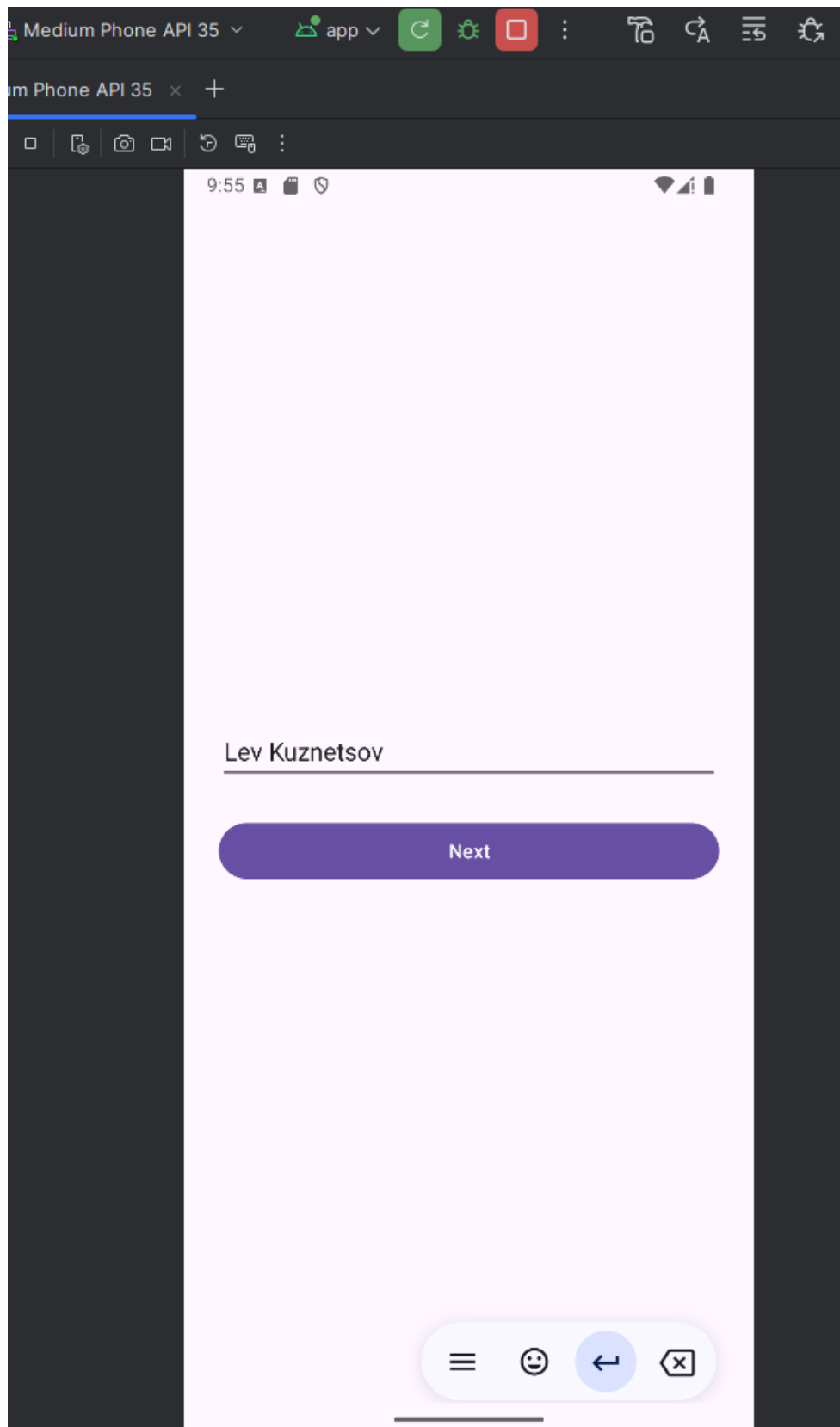


Рисунок 2.4.1 – Приветственная страница приложения

На рисунке 2.4.1 представлена приветственная страница приложения.

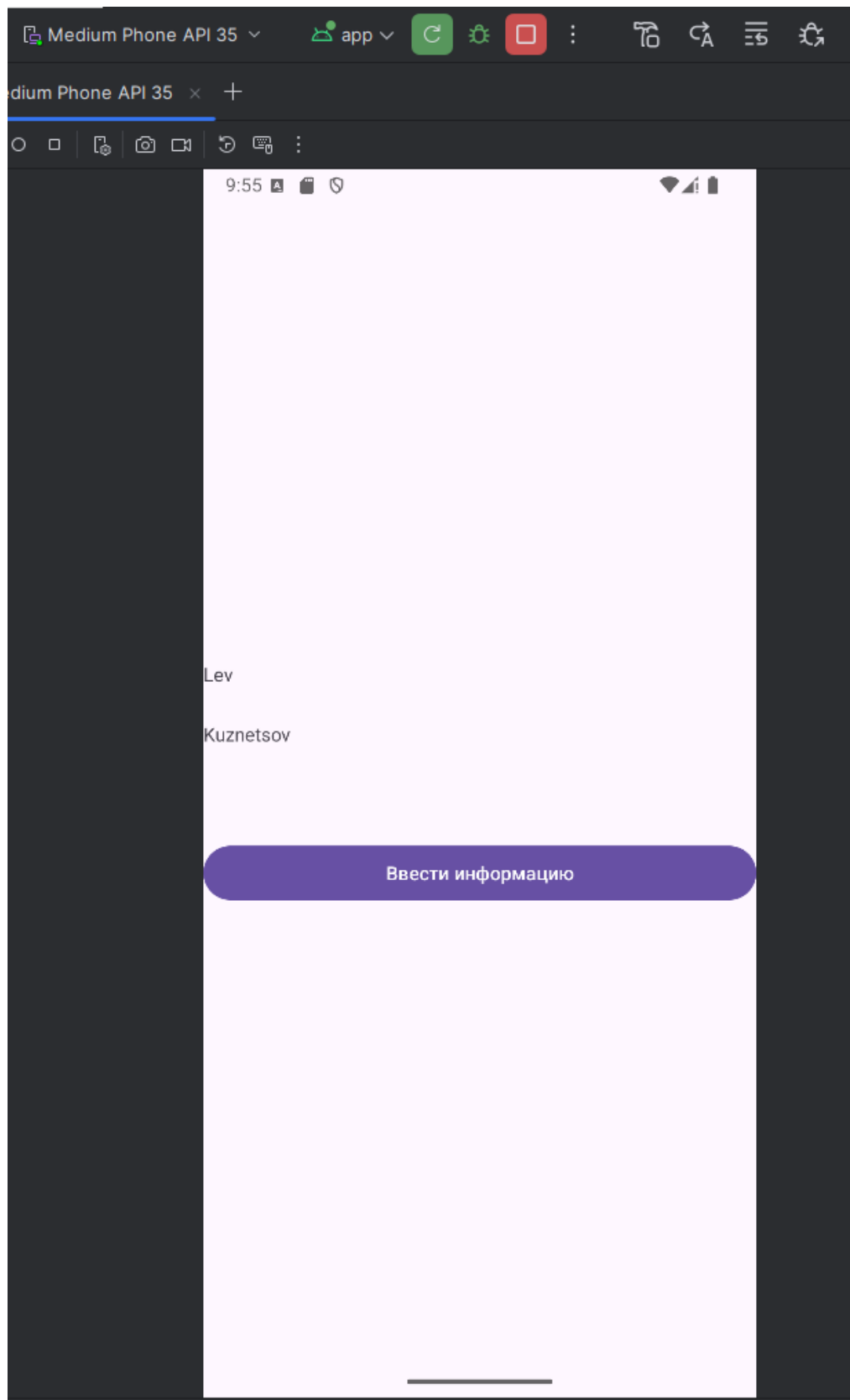


Рисунок 2.4.2 – Отображение 2 activity приложения

На рисунке 2.4.2 происходит отображение ранее введённых данных, а также возможность на переход activity с вводом необходимых данных.

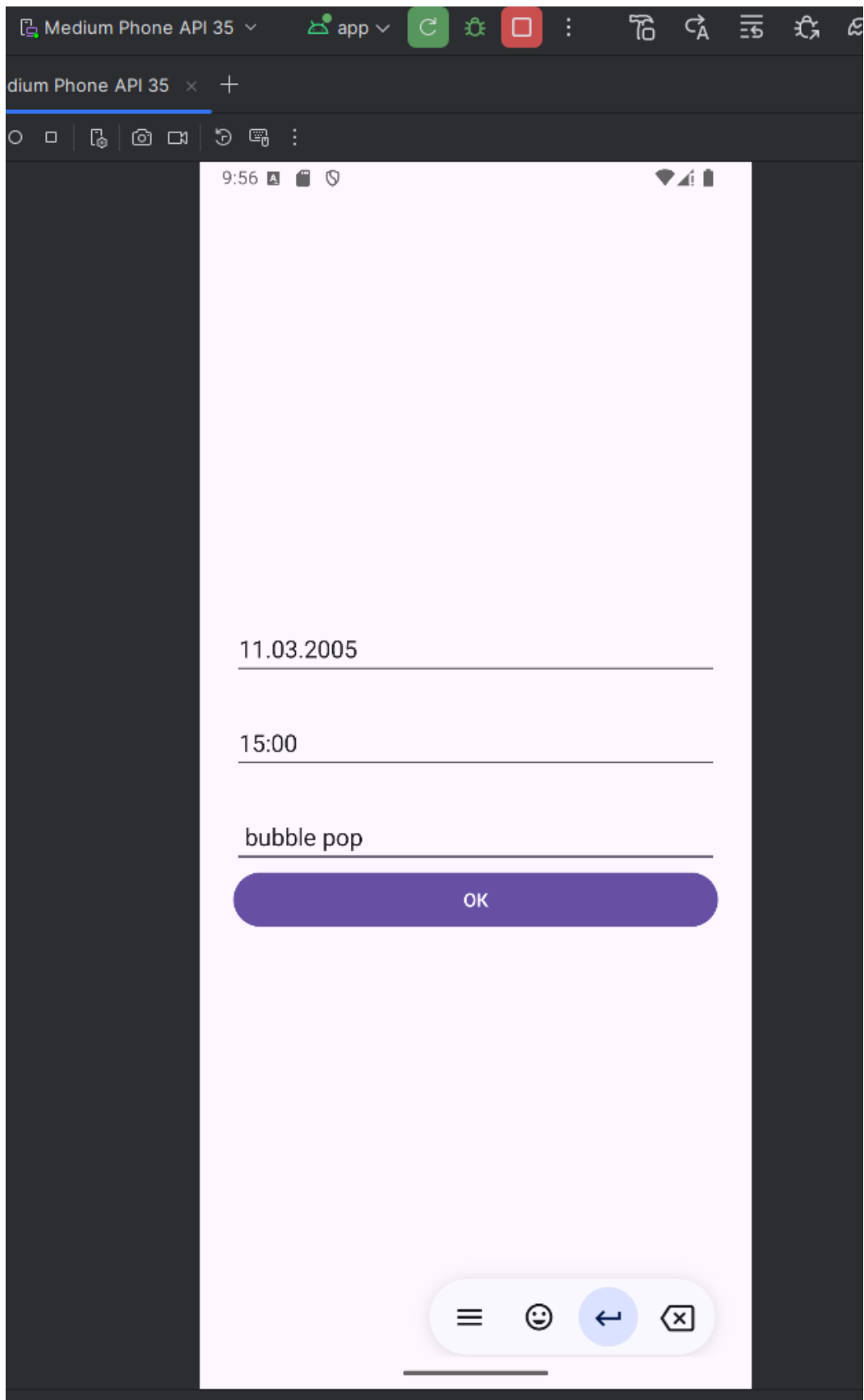


Рисунок 2.4.3 – Последняя activity приложения

Рисунок 2.4.3 отображает activity с вводом различных данных, а также кнопку с возможностью подтверждения данных.

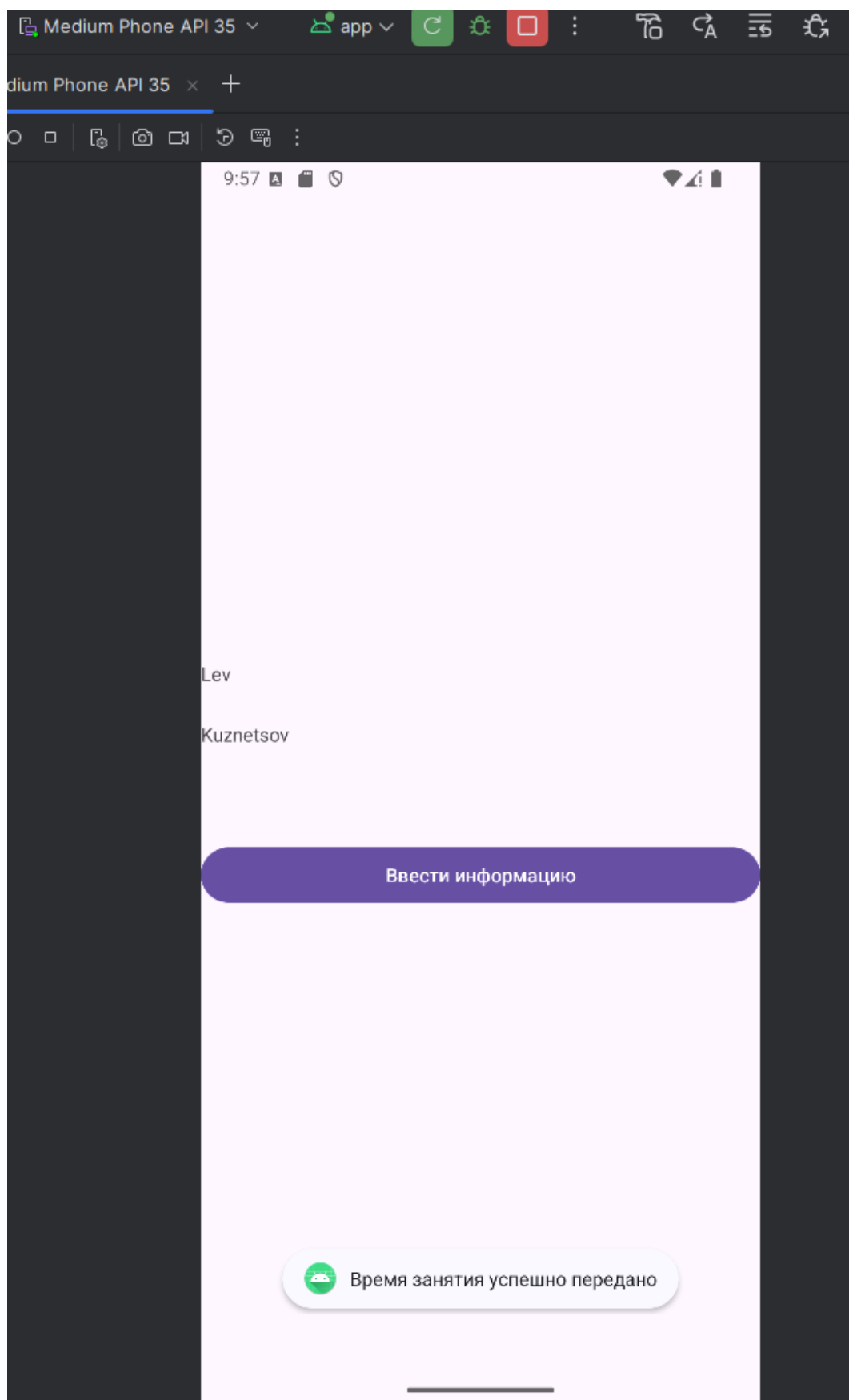


Рисунок 2.4.4 – Оповещение об успешной передачи данных

На рисунке 2.4.4 отображено подтверждение успешной отправки ранее введённых данных.

2.5 Создание проекта

Создаём проект под названием “Practic4.1” согласно поставленной задаче (рисунок 2.5.1).

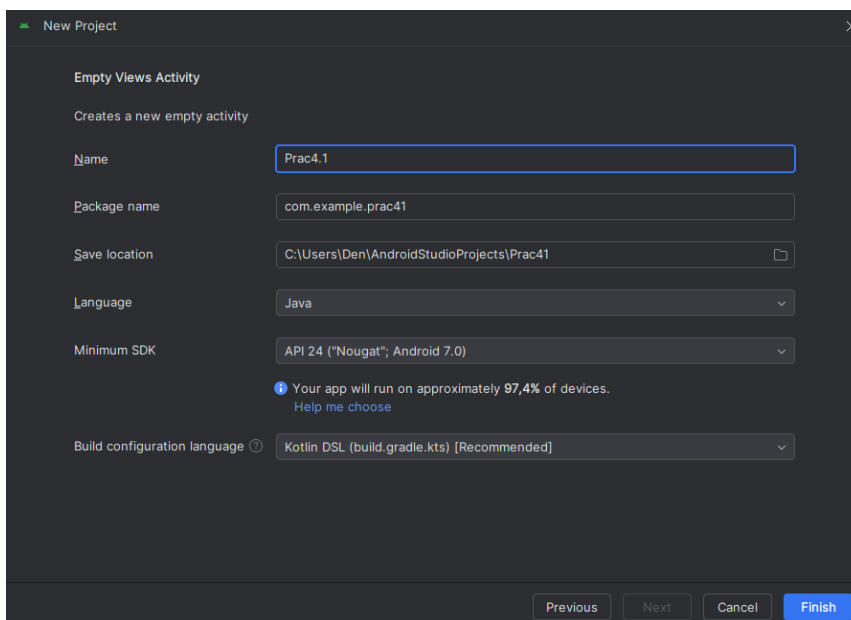


Рисунок 2.4.1 – Создание проекта

2.5.1 StaticFragment

Перед добавлением фрагмента необходимо его создать (рисунок 2.5.1.1).

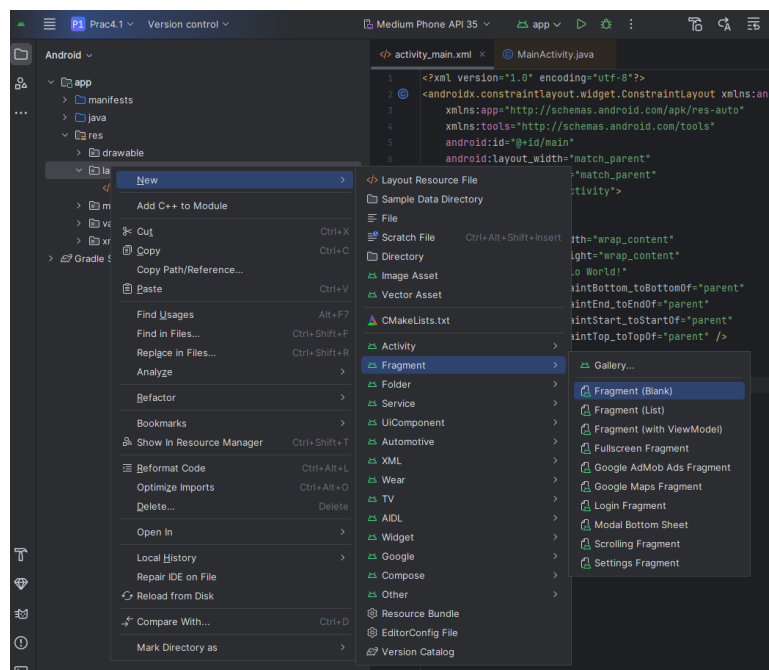


Рисунок 2.5.1.1 – Создание фрагмента в Android Studio

В данном случае фрагмент будет создан статически (рисунок 2.5.1.2).

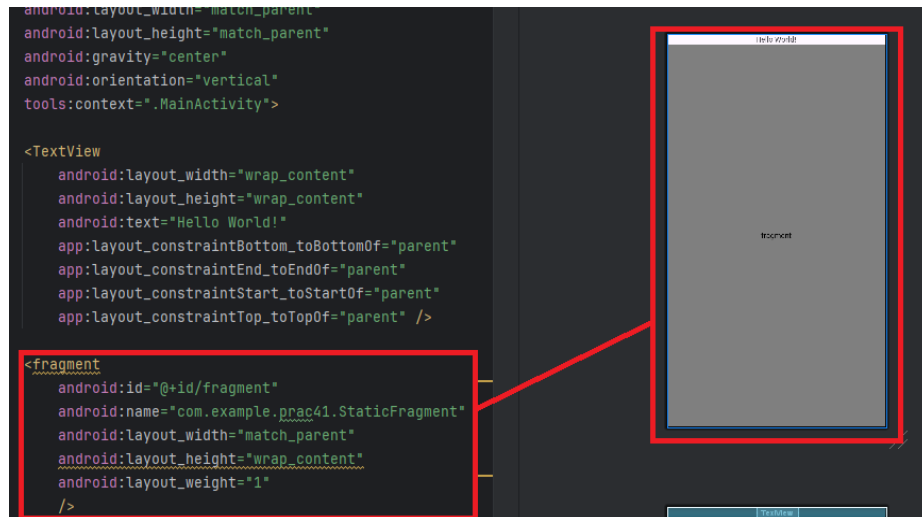


Рисунок 2.5.1.2 – Статический способ создания фрагмента

Для статического создания фрагмента достаточно объявить фрагмент в файле разметки со всеми необходимыми характеристиками.

2.5.2 DynamicFragment

Далее был динамически создан новый фрагмент (рисунок 2.5.2.1).

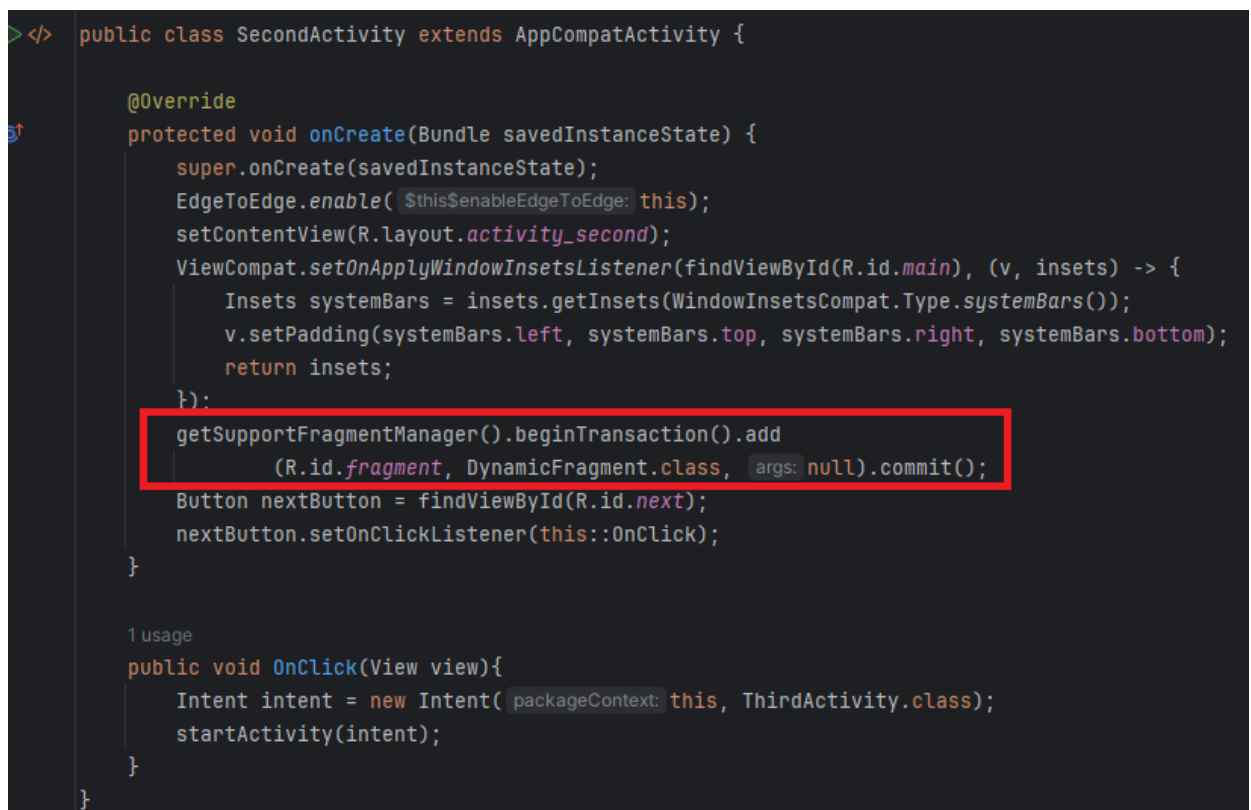


Рисунок 2.5.2.1 - Динамический способ создания фрагмента

Данный способ создания фрагментов является наиболее распространённым, так как именно для динамического изменения расположения элементов в приложении больше всего подходят фрагменты.

2.5.3 LastFragment

И последний способ создания фрагментов - через `fragmentContainerView` (рисунок 2.5.3.1).

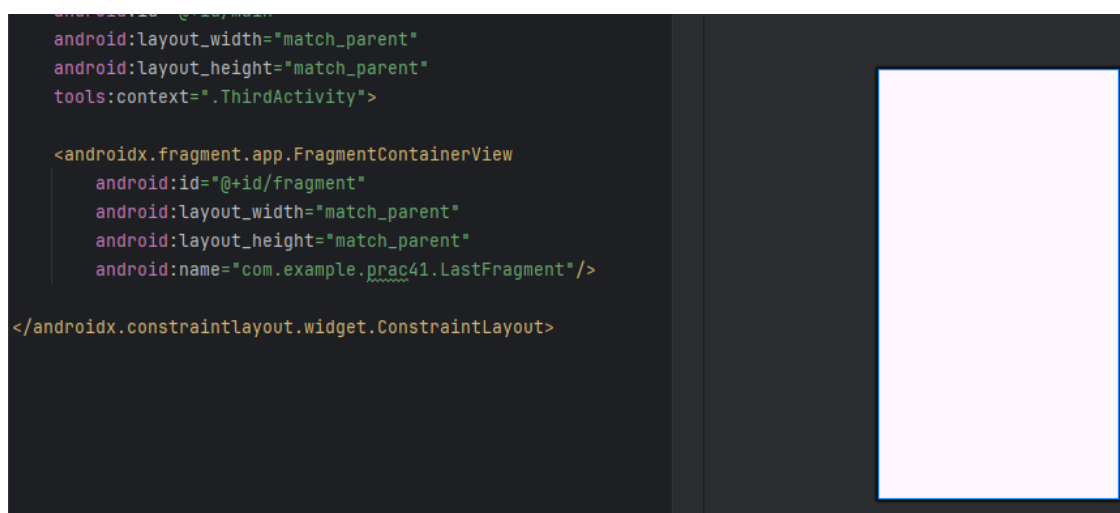


Рисунок 2.5.3.1 – Создание фрагмента при помощи `fragmentContainerView`

В данном случае сам же фрагмент не отображается (рисунок 2.5.3.2).

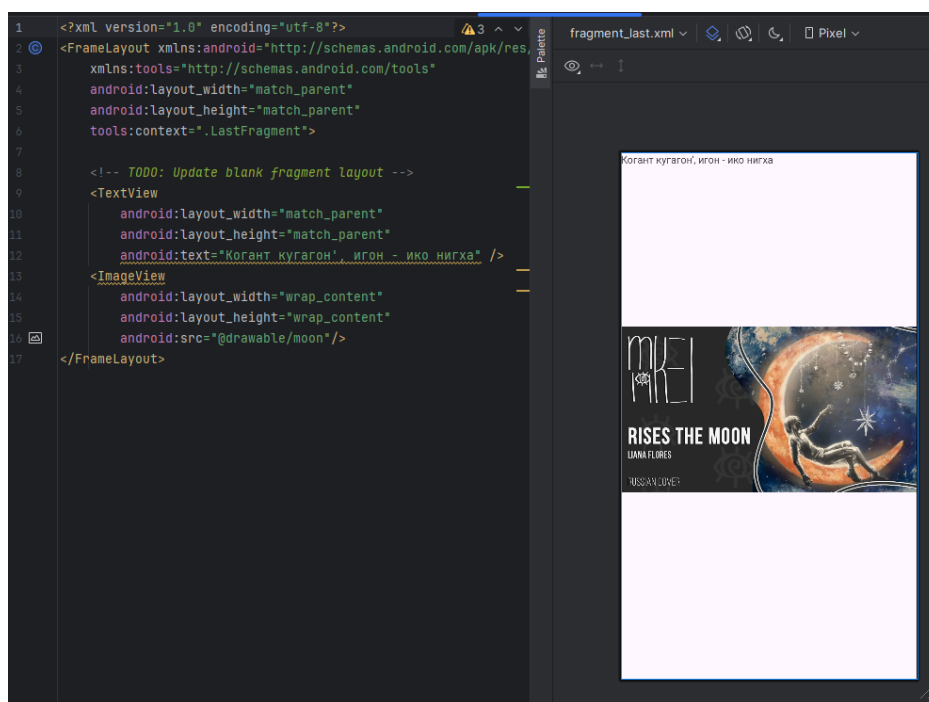


Рисунок 2.5.3.2 – Кодовый и графический вид добавляемого фрагмента

Далее отобразим конечный вид приложения (рисунки 2.5.3.3 – 2.5.3.5).

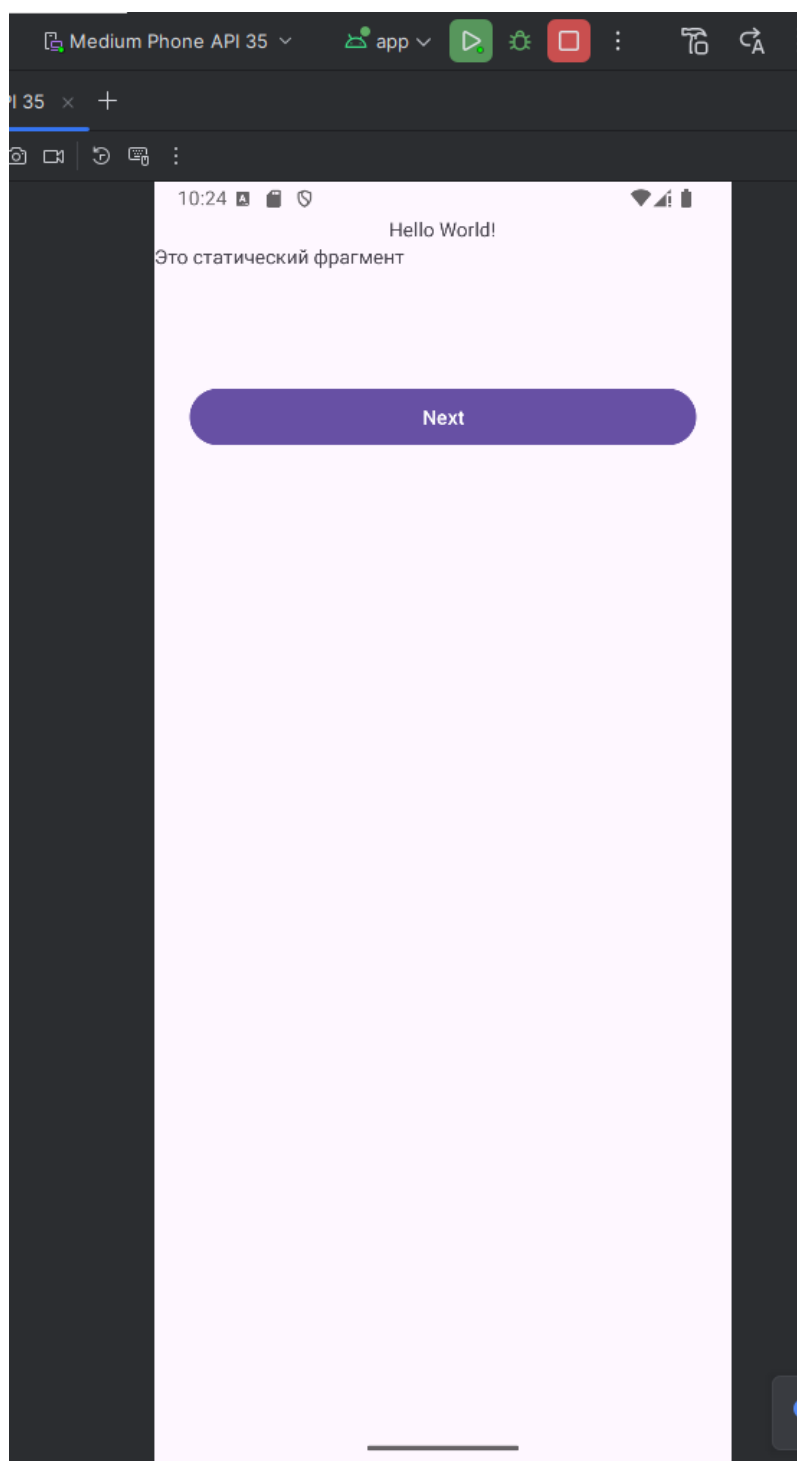


Рисунок 2.5.3.3 – Часть 1 отображения итогового вида приложения

На рисунке 2.5.3.3 показано отображение статического добавления фрагмента.

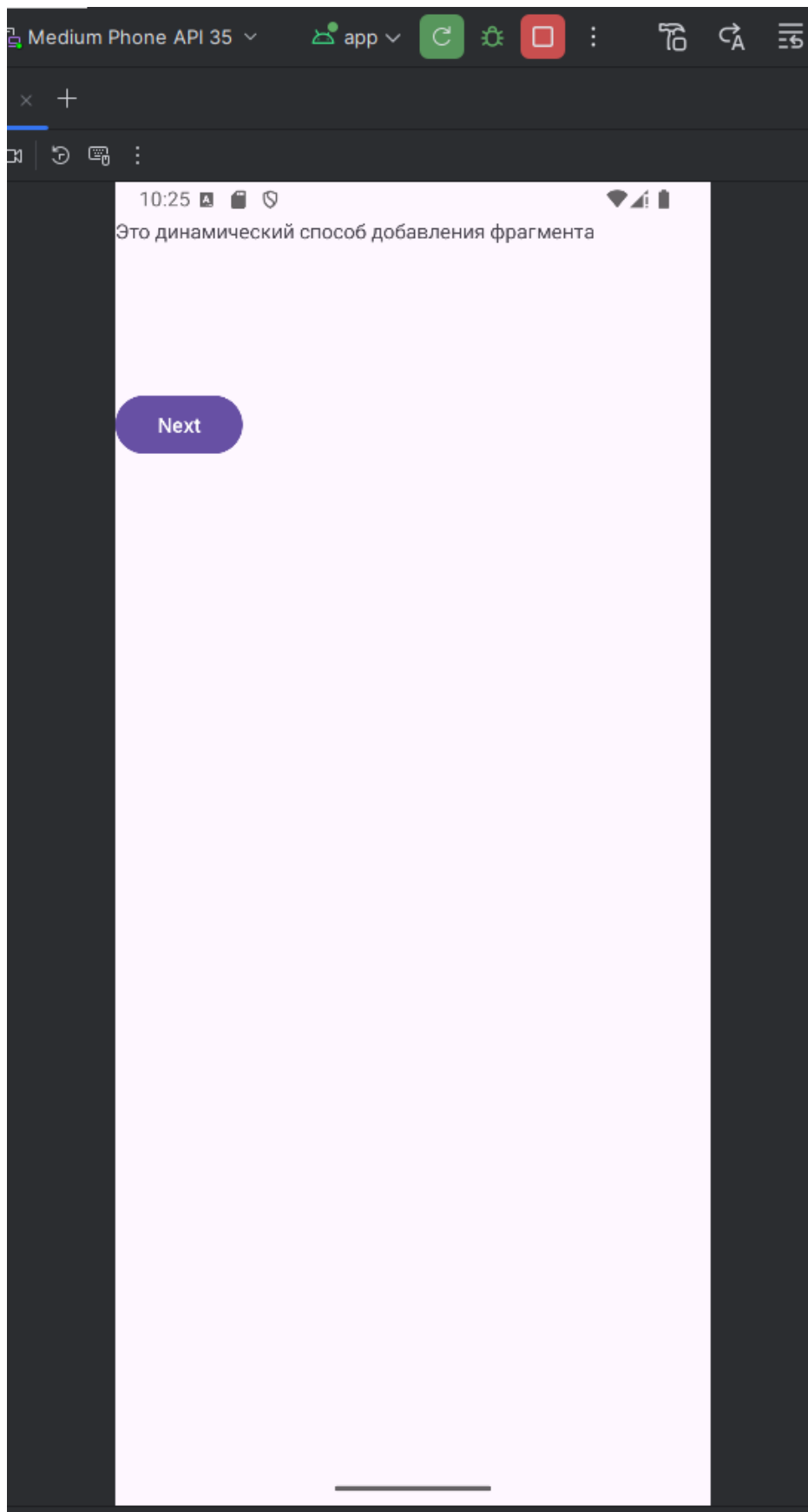


Рисунок 2.5.3.4 – Часть 2 отображения итогового вида приложения

На рисунке 2.5.3.4 показано отображение динамического добавления фрагмента.

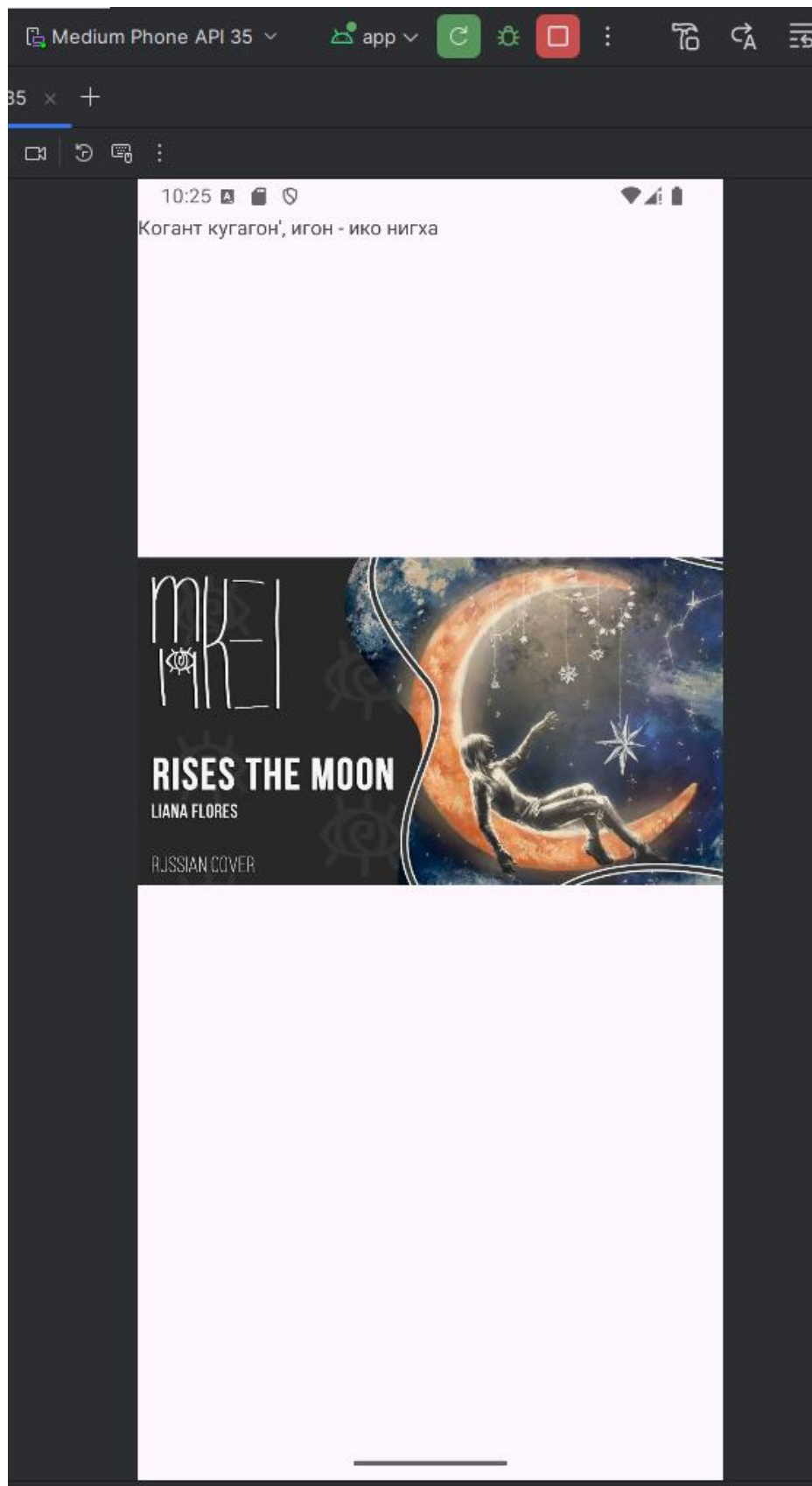


Рисунок 2.5.3.5 – Часть 3 отображения итогового вида приложения

На рисунке 2.5.3.5 показано отображение добавления фрагмента через `fragmentContainerView`.

ЗАКЛЮЧЕНИЕ

В ходе данной работы было проведено ознакомление как с различными видами фрагментов, так и со способами их добавления в итоговое приложения. Полученные знания были закреплены путём выполнения практического задания.